DATALOGGER

Indlejrede systemer, Programmering & Projektstyring og forretningsforståelse.

Zealand Erhvervsakademi

18. December, 2025



1. Semester Hold A

Vejleder: Palle Zych, Tomasz Blaszczyk & Zuhair Haroon Khan.

Gruppe: [navn]

[navn], [navn]

[navn] & Lea Mette Jacobsen.

Antal Tegn: 47,842

Billede Kilde: https://chatgpt.com/ Billedet er generet af Ai.

Indholdsfortegnelse:

1. Introduktion: [navn] ***3***

1.1 Interessent Analyse: [navn] 4

2. Problemformulering: [navn] ***5***

3. Metode: Lea ***6***

3.1 SCRUM: 6

3.2 Teknisk Metode (IoT): 10

4. System Overview: ***15***

4.1 Formål af systemet: [navn] 15

4.2 Software Overview: [navn] 17

4.3 System Architecture: [navn] 27

5. Analyse: ***28***

Dataanalyse: Lea **28**

Produkt analyse: Lea **37**

Fejlkilder: [navn] **37**

Kvaliteten af komponenterne: 37

Photo resistor: 38

Simulering af bikube: 38

6. Konklusion: Lea ***38***

7. Videreudvikling: **39**

8. Evaluering af samarbejde: Alle ***40***

Kildeliste: [navn] ***41***

Bilag: ***43***

A. Sprint Backlog: 43

B. Gruppekontrakt: 45

C. Kildekode (.ino): 46

D. Software Overview 51

E. Data (.csv) / Excel Table: 51

1. Introduktion: [navn]

Bier spiller en afgørende rolle i både landbrug og miljø; de er nødvendige for pollinering, honningproduktion og vedligeholdelse af biodiversitet.[1] Projektets mål er at designe en IoT-baseret datalogger, der vil overvåge og identificere ændringer i miljøvariabler, der anvendes til at vurdere biers sundhed og velfærd indenfor og udenfor bistadet. De miljøvariabler, vores IoT-produkt vil overvåge, er temperatur, fugtighed, lysintensitet og tid; disse variabler gemmes lokalt i dataloggeren på et SD-kort. IoT-dataloggeren vil derfor give kvantitative data til biavlerne for bedre at forstå forholdene, korrekt vurdere biernes velbefindende og bestemme, om der skal foretages manuelle ændringer.

Produktet vil give en klar sammenligning mellem forholdene inde i og uden for bistadet for at vurdere, om de har de rette betingelser, og muliggøre bedre håndtering fra biavlernes side, med overvågning over en periode på 3 dage og tidsstempler hver 10. minut. Denne proces tillader sporing af forholdene i løbet af dagen; ved at bruge et realtidsur (RTC) sikres det, at alle målinger af miljøfaktorer nøjagtigt er markeret med tiden for at se, hvornår ændringer sker i bistadet.

Målet er identifikation af tendenser og ændringer over tid og vurdering af, om forholdene lever op til standarden for den rette sundhed og velfærd for, at bierne kan trives; IoT-dataloggeren giver denne indsigt til biavleren. Indsigt vil give en klarere forståelse af de miljøvariabler og hvordan de interagerer med bierne.

De følgende sektioner beskriver metoden, systemdesignet, dataindsamlingen og analysen, der er nødvendige for korrekt at evaluere IoT-dataloggeren og bestemme, om den opfylder sine mål.

[1] Derrick, Scott. “The Importance of Beehives in Nature for Our Ecosystems and Biodiversity.” *Blythewood Bee Company*, 23 Mar. 2025, blythewoodbeecompany.com/blogs/news/beehive-in-nature. Accessed 4 Dec. 2025.

1.1 Interessent Analyse: [navn]

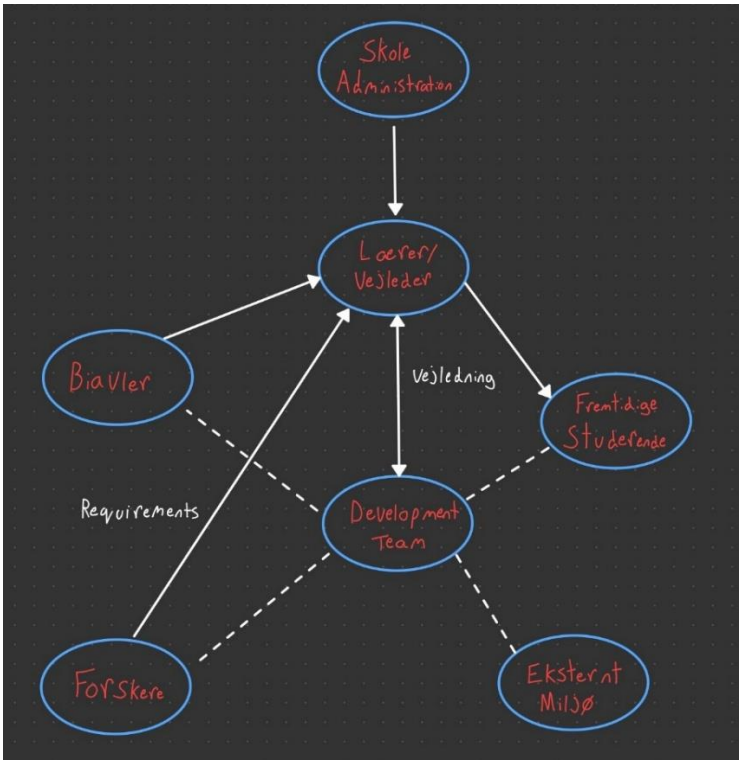
De primære interessenter, som IoT-systemet prioriterer, er biavlerne, miljøforskere og potentielle studerende, der lærer om IoT. Disse interessenter værdsætter dette produkt, fordi det giver dem kvantitativ information og værdifulde data om IoT og miljøforholdene i bistadet.

Biavlerne og forskerne er de primære interessenter, da de data, som IoT-systemet vil indsamle og overvåge, er de mest værdifulde for dem, fordi det giver en klar forståelse af, hvordan tendenser og ændringer i de afhængige miljøvariabler inde i og uden for bistadet interagerer med biernes velfærd og miljøforhold. Dette produkt kan direkte give værdifuld information til at støtte vurderingen af velfærden og miljøforholdet i bistadet.

Skoleadministration og lærerne er sekundære interessenter, der har iværksat denne undersøgelse og vil give den akademiske ramme. Udviklingsteamet har til opgave at skabe det nødvendige produkt, der vil tilfredsstille alle interessenter.

Der er også potentielle eksterne interessenter, der kunne påvirke vores udvikling, såsom Apis RAM, som er en agentbaseret kolonimodel, der giver en ramme til at udforske effekter som pesticider eller andre stressfaktorer på honningbi-kolonier.[2] Dette kunne potentielt tjene som en forskningsreference for, hvordan man tolker miljødata, men interagerer ikke direkte med udviklingen af IoT-systemet.

[2] Duan, Xiaodong, et al. "ApisRAM Formal Model Description." *EFSA Supporting Publications*, vol. 19, no. 2, Feb. 2022, https://doi.org/10.2903/sp.efsa.2022.en-7184.



Figur 1.1 - Sociodiagram

2. Problemformulering: [navn]

Problemformuleringen, der er relevant for undersøgelsen, skal fokusere på det centrale problem, som IoT-produktet skal løse. Dette skal omfatte evnen til at indsamle data fra sensorerne på afstand og gøre dette kontinuerligt for at få pålidelige og valide data, der gør det muligt at finde de stabile og sunde forhold for bierne i bistadet. Problemformuleringen er:

Hvordan kan et IoT-system designes til at måle og karakterisere de miljøforhold, der er nødvendige for at opretholde et sundt og stabilt bistade, og hvordan kan sensordata anvendes til at opdage potentielle udsving, der kan påvirke disse optimale forhold?

3. Metode: Lea

I dette afsnit vil vi gennemgå planlægning og udviklingen af vores datalogger. Vi kommer ind på arbejdsprocessen med en gennemgang af SCRUM og hvordan vi har arbejdet med dette.

Efterfølgende dykker vi ned i planlægningen og udviklingen af vores produkt, og de overvejelser vi har haft undervejs.

3.1 SCRUM:

Udviklingen af et IT-produkt er en kompleks størrelse, med tværfagligt samarbejde og mange faktorer der kan ændre processen og visionen for et produkt. For at rumme disse faktorer bruger vi SCRUM som planlægningsredskab.

SCRUM er en dynamisk tilgang der sætter klare rammer for samarbejdet i et team. Rammerne definerer roller, møder og arbejdsopgaver. Mellem møderne arbejder et development-team i sprint.

SCRUM-metodens arbejdsgang ses på figur 3.1. Projektet starter med en vision, der oprettes en backlog af prioriteret arbejdsopgaver. Disse løses i sprint, efterfulgt af opfølgning og evaluering. Sprint gentages til deadline for projektet. Slutteligt holder man et retrospektivt møde, hvor man vurderer forløbet.



Figur 3.1 Overblik over et typisk scrum forløb

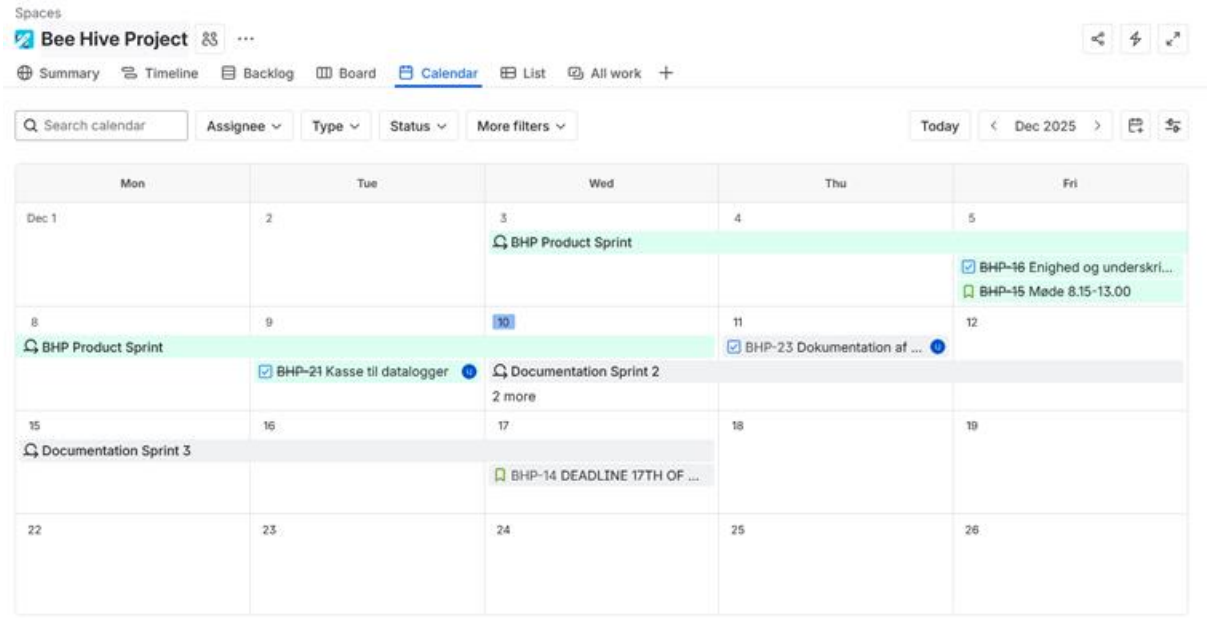
Gruppens forløb:

Vi startede med at forventningsafstemme og oprette en gruppekontrakt [BILAG], der danner grundlag for vores samarbejde. I denne gruppekontrakt uddeler vi også rollerne i vores SCRUM-team. Projektets vision bliver givet af en **product-owner**. Da projektet er en eksamensopgave, er vores underviser product-owner. Det er skiftende hvor meget product-owner er med ind projektet. I dette projekt overlader product-owner de fleste planlægningsopgaver til vores SCRUM-team.

Vores **Scrum master** er [navn] Rollen består i at være tovholder og at rammerne bliver overholdt. Han har oprettet og holdt overblikket over backlog, prioriteret opgaverne og sikret sig at de blev fordelt. Dette har han gjort i samarbejde med development-team, så vi bedst udnytter vores styrkerne.

Vi har alle været en del af **development team**. Denne rapport er skrevet i fællesskab og alle gruppens medlemmer har været med til at designe dele af dataloggeren.

For at holde overblikket over projektet har vi benyttet softwaren JIRA. JIRA er et project managing tool med mange muligheder. Vi har benyttet den til at kan oprette en projektmappe med sprint backlogs, uddelegere opgaver og danne os et overblik. Projektmappen kan tilgås af alle i teamet. JIRA giver også adgang til en kalender, hvor sprint og deadlines kan indsættes (fig. 3.2)



Figur 3.2: Kalenderoverblik fra JIRA.

Vi har i forløbet holdt to daglige møder. Vi startede dagen med et morgenmøde, med en kort opsamling og gennemgang af dagens plan. Efter arbejdsdagen holdt vi et eftermiddagsmøde hvor opgaverne og dagens arbejde evalueres. Hvis det er nødvendigt, har vi ændret planen. Det gjorde vi f.eks. med opsætningen af vores datalogger. Vores intention ved start-møde d. 9/12, var at sætte dataloggeren i bikuben på Zealands. Vi ønskede dog mere sikkerhed i at det virkede og ville gerne have den under opsyn, så vi besluttede at sætte den op hjemme hos et gruppemedlem.

I dette projekt har vi arbejdet i tre sprint. Vores overordnet mål har været at:

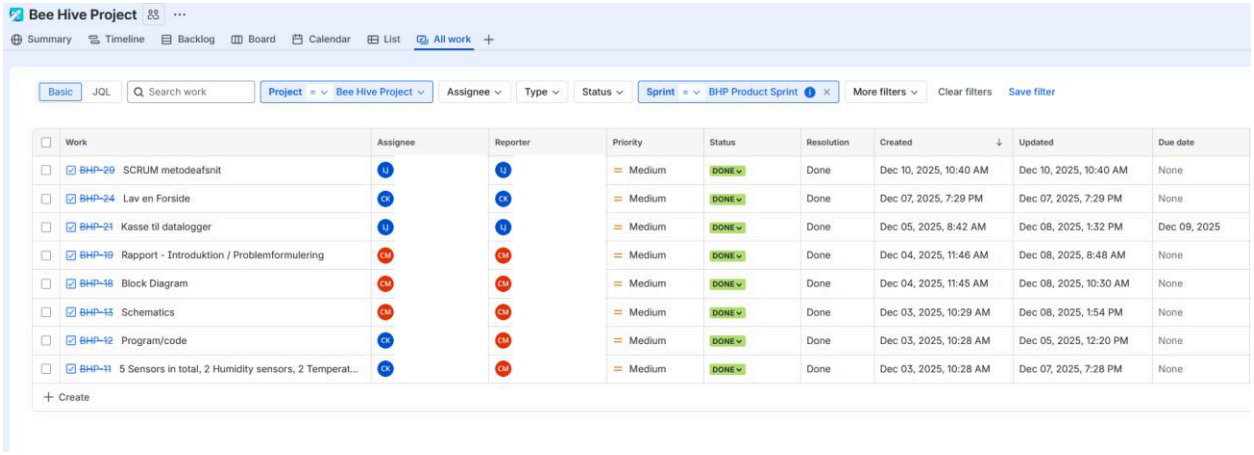
- Design datalogger produkt
- Opsætte datalogger og lave analyse af data
- Dokumentere løbende og aflevere samlet rapport

Vores samlet backlog kan læses i bilag A.

Sprint 1:

I dette sprint fik vi overblik rapportens omfang og opsatte en skabelon for den i et fælles dokument. Dele af rapporten påbegyndes. Dataloggeren blev samlet og programmeret. Den bliver løbende testet og processen dokumenteres.

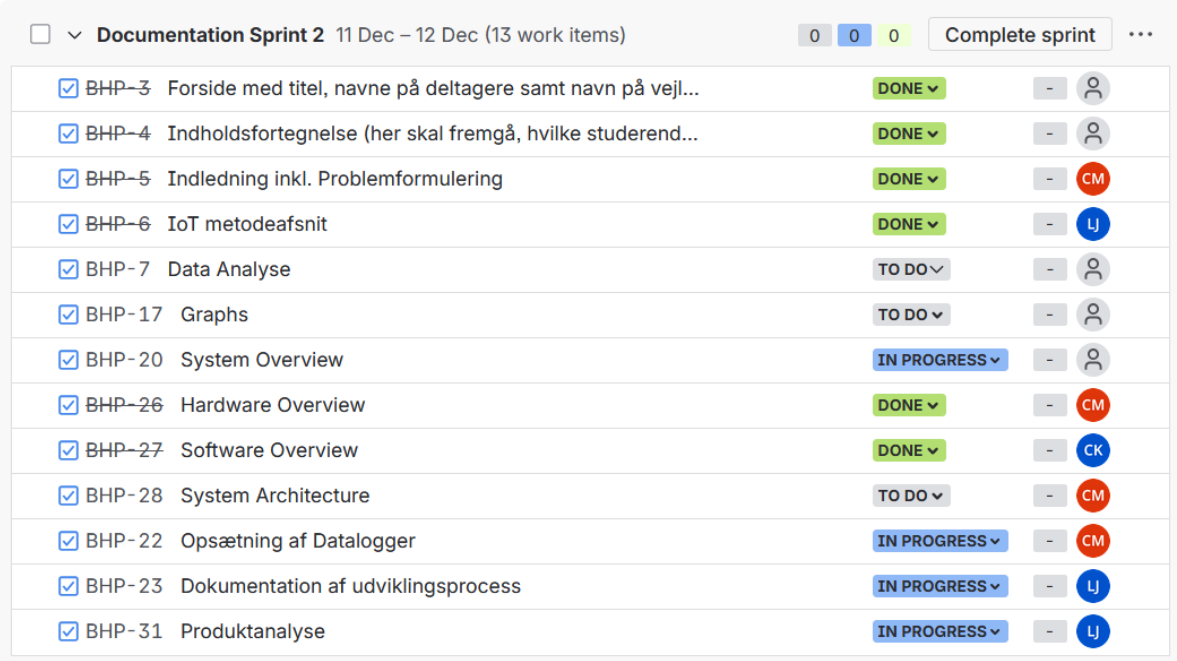
Her ses vores første sprint, opsat i JIRA.



Figur 3.3 - Sprint 1

Sprint 2:

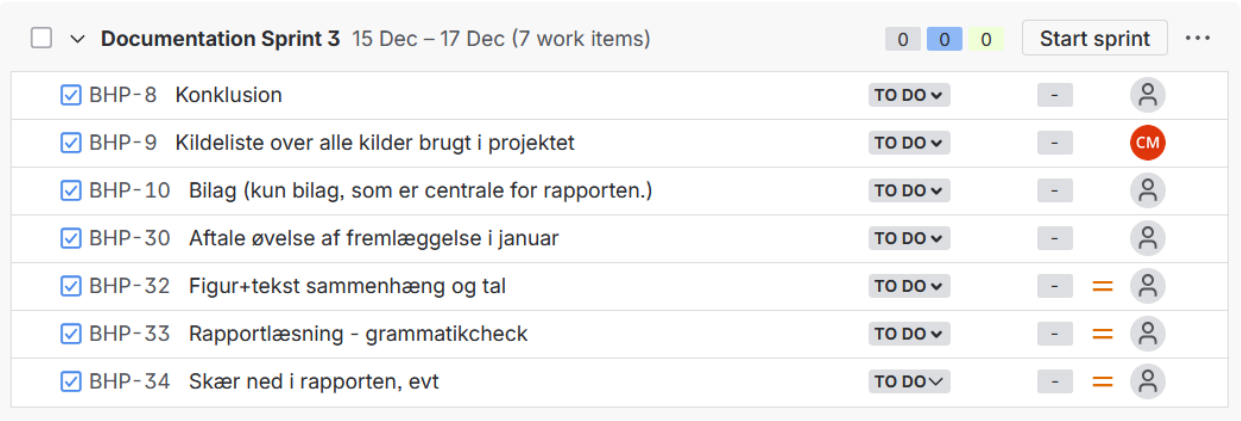
Dataloggeren opsættes og vi indsamler data over 3 dage. Imens vi samler data, arbejder vi på rapporten. Når dataloggeren er færdig, analyserer vi resultaterne og dokumenterer færdigt.



Figur 3.4 - Sprint 2

Sprint 3:

I det sidste sprint lavede vi konklusionen på vores rapport og tjekkede at alle tekniske detaljer var i orden. Vi lavede en afsluttende vurdering af rapporten og afleverede. Efterfølgende holdt vi et sidste eftermiddagsmøde, hvor vi aftalte dato for at øve fremlæggelse.



Figur 3.5 - Sprint 3

Opsamling på SCRUM:

SCRUMs fleksible rammer har gjort det muligt for os løbende at vurdere vores produkt. Den iterative arbejdsgang med skift mellem møder og opgaver giver et godt arbejdsflow og god kommunikation. De korte møder ligger også op til idé-deling og feedback, som kan gavne produktet. De gentagende test af produktet sikrer kvaliteten og funktionaliteten.

3.2 Teknisk Metode (IoT):

Formålet med produktet, er at samle data i og omkring bikuben. Denne data skal bruges til at karakterisere miljøet omkring vores simuleret bikube. Herfra kan vi sammenligne med gældende viden om biers behov og vurdere om det er et godt miljø for bier at bo i.

Ud fra produktkravene har vi designet og produceret en datalogger. Her gennemgås processen, fra udvælgelse af komponenter til opsætningen af dataloggeren. Den skematiske opsætning af dataloggeren er at finde under afsnittet ”system architecture”.

Udviklingsproces:

Vores datalogger skal stå udenfor, og beskyttes mod vejret. Vi har valgt at bruge en papkasse (Fig 3.6). Den er sat sammen med sort gaffatape og har fået et lag gaffetape i bunden for at beskytte mod vand. Vi valgte at have flere huller til kablerne, men vi kunne godt have gået med ét hul til alle kabler. Disse er efterfølgende blevet tapet over, for at beskytte yderligere.



Figur 3.6 – Beskyttelseskasse til datalogger.

Komponenter:

Formålet med vores datalogger er at den skal opsamle data indeni og udenfor en bikube. Vi skal bruge data på tid, temperatur og fugtighed i de to miljøer og lysintensitet. Denne data skal gemmes på et SD-kort. Hele systemet sættes sammen på en micro-controller af typen Arduino Megaboard 2560. Softwaren vil blive gennemgået i et senere afsnit.

Vi bruger følgende komponenter:

- Arduino Megaboard 2560
- RTC – Registrer tiden
- DHT – En temperatur og fugtmåler. Denne bruger vi to af.
- Fotomodstand (LDR) – måler lysintensiteten uden for bikuben
- Hukommelses SPI – Giver mulighed for at tilføje SD-kort til oplagring af data
- Et breadboard – til at samle kredsløbet

Arduino Megaboard 2560

Vores hovedkomponent er et Arduino megaboard 2560. Dette er en micro-controller fra firmaet Arduino. Vi bruger den til at sætte vores integreret system op. Det er nemt at tilgå, har sit eget programmeringsprogram af samme navn, og en stor brugerbase som deler viden og inspiration på online.
Boardet fungerer på temperaturer mellem –40 og 85 C, så dataindsamling udendørs er sikkert.
Den har ikke godt af at blive våd, derfor skal den beskyttes mod vand og høj fugt.
Boardet har 54 digitale pins og 16 analoge input pins.

- **Flash-hukommelse:** 256 KB, heraf 8 KB reserveret til bootloader
- **SRAM:** 8 KB
- **EEPROM:** 4 KB til lagring af vedvarende data
- **Clockfrekvens:** 16 MHz
- **Strømforsyning:** 5V via USB eller ekstern forsyning (7-12V via VIN-pin)
- **USB-tilslutning:** USB Type-B port til programmering og seriel kommunikation
- **Kommunikation:** Understøtter UART, I2C og SPI protokoller
- **USB Chip Interface:** ATMEGA16U2

RTC

Denne komponent registrer tid og dato for vores måling af temperatur, fugtighed og lys. Vores model er en *RTC DS3231*. Den har en lav fejlmargin så længe dens temperatur ligger mellem 0-40 ˚C. I det tidsrum hvor vores datalogger har været samlet data, registrer den temperatur mellem [x-y]. Vi vurderer derfor ikke at temperatursvinginger har haft stor inflydelse på vores RTC data.
RTC kræver et CR2032 batteri, som vi har indsat.

DHT

Dette er en sensor, som måler luftfugtighed og temperatur. Luftfugtigheden måles relativt og kan være fra 0-100 % med en 2-5 % fejlmargin. Temperaturmåleren kan måle fra -40 ˚C til 80 ˚C og har 0.5 ˚C i fejlmargin. Sensoren konverterer selv til digitalt signal, så den er nem at arbejde med. Vi har opsat to sensorer, jvf. FIGURX.

Vi har udstyret begge vores DHT-sensorer med en sort plastik strips, da de sad løst. Vi testede dem efterfølgende og vurderer at de fungerer fint.

Fotomodstand (LDR)

Vi vil gerne kende til varmepåvirkningen af miljøet i bikuben, og måler derfor lyset den er udsat for. Til dette bruges en fotomodstand, også kendt som en LDR. Den fungerer ved at sænke modstanden, når den udsættes for lys.

Hukommelses SPI

Den data vi indsamler, ønsker vi at gemme i en .csv fil. Til dette bruger vi en hukommelses SPI, som er en komponent der bruges til at tilslutte et SD-kort. Arduino programmeres til at oprette en .csv fil og lagrer vores data her.

SD-Kort

Et SD-Kort (Secure digital-kort) er en flytbar lagreenhed, der bruges til at lagre og overfører digitale data. SD-Kortets lagerkapacitet varierer, men i dette projekt er der blevet brugt et SD-Kort på 4GB, hvilket er mere end rigeligt lagerplads til vores behov.

Kredsløbssystemet

For at samle kredsløbet, har vi opsat vores system i et breadboard med dupontwires (Fig 3.7). I testfasen brugte vi dupontwires med stik på, da de er nemme at rykke rundt på. Vores testfase skiftede mellem implementering af hardware og test af softwaren. Sammenspillet mellem disse, kan læses i afsnit 4. Systemoverview.

Efter testfasen skiftede vi alle wires ud, med nogle der var tykkere og sad bedre fast i vores breadboard. De to sensorer der skulle måle udenfor, valgte vi at lodde fast og herefter dække lodningerne med tape.

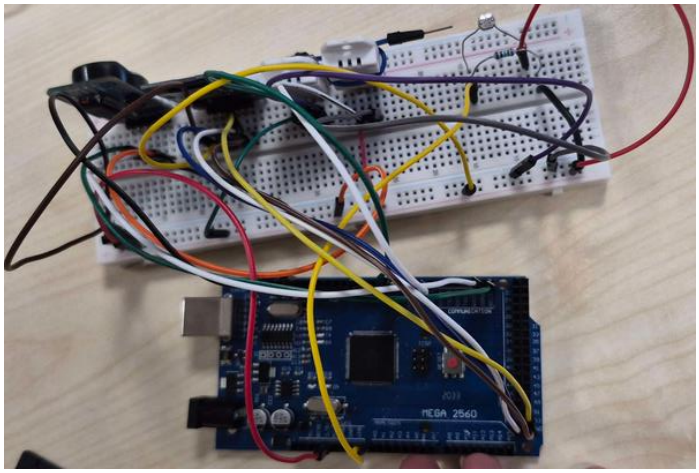


Fig 3.7: Opsætning af datalogger til test af kredsløb og kode.

Dataopsamling

Efter test og udskiftning af wires, sætter vi vores datalogger op. For at simulere en bikube, ligger vi dataloggeren i en trækasse, og sætter en trækasse ovenpå som låg (Fig. 3.8). Der måles temperatur og fugtighed inden i 'bikuben'. Udenfor måles der temperatur, fugtighed og lys.

Kassen er udstyret med huller til de sensorer som skal måle udenfor. Disse blev tapet over, efter at sensorerne er ført igennem. Vi valgte at ligge et håndklæde mellem de to kasser, pga. regn. Dette var en dårlig idé, men mere herom i fejlkilder.

Vores datalogger får strøm fra en mobil-oplader der er tilsluttet en kabeltromme. Mobilopladeren sikrer at vores datalogger ikke modtager mere strøm end den kan holde til.



Fig 3.8: Opsætning af dataloggeren udenfor i en trækasse.

Under dataindsamling har vi løbende tjekket at dataloggeren virker. Vi vælger også at fjerne håndklædet der var lagt mellem kasserne, da det udgjorde en fejlkilde.

Efter dataopsamling pakkede vi dataloggeren sammen og gennemgik vores data.
Vi har opsat vores data i en Excel fil (se figur 3.9), ved brug af vores kode. I vores Excel fil har vi valgt at der skulle være en overskrift (headeren), hvor sensorernes afmålinger skulle ligge under de passende overskrifter. Dette har vi valgt, da det giver mere struktur og gør det nemmere for os selv, når dataene skal analyseres. Derudover har vi lavet, så der er et specifikt interval (hver time) til, hvornår overskriften skal komme igen. Dette har vi valgt at gøre, for nemmere at kunne overskue vores lange datasheet.

Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10-12-2025	17:19:57	101C	862%	96C	1000%	0
10-12-2025	17:29:57	101C	863%	97C	1000%	0
Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10-12-2025	18:19:57	102C	873%	100C	989%	0
10-12-2025	18:49:57	104C	875%	100C	1000%	0
10-12-2025	18:59:57	104C	876%	100C	997%	0
Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10-12-2025	19:19:57	104C	877%	101C	991%	0
10-12-2025	19:29:57	105C	878%	101C	988%	0
10-12-2025	19:49:57	104C	877%	100C	997%	0
10-12-2025	19:59:57	104C	880%	100C	998%	0
Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10-12-2025	20:19:57	104C	881%	101C	999%	0
10-12-2025	20:29:57	105C	884%	102C	993%	0
10-12-2025	20:39:57	105C	884%	101C	993%	0
10-12-2025	20:49:57	105C	886%	102C	986%	0
10-12-2025	20:59:57	106C	888%	102C	985%	0
Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10-12-2025	21:19:57	105C	885%	100C	988%	0
10-12-2025	21:59:57	104C	889%	100C	985%	0
Dato	Tid	Temp_IND	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET

Figur 3.9 – Uddrag fra vores Excel fil. Headers opstår løbende i datafilen for overblik.

Efter download af vores data, er den blevet sorteret, behandlet og analyseret. Data analysen kan ses under afsnit 5. Analyse.

4. System Overview:

4.1 Formål af systemet: [navn]

Målet med systemet er at overvåge miljøforhold, specifikt fugtighed, temperatur og lysintensitet. Dette betyder, at følgende sensorer (DHT, LDR) er essentielle for systemet. Systemet sigter mod at indsamle data over en periode på tre dage, både indeni og udenfor bistadet. Ved at bruge en RTC kan vi indstille et internt ur, som vil registrere og indsamle tilstrækkelig kvantitativ information hvert 10. minut. Disse data indsamles og gemmes i et Micro SD SPI-modul, som vil indeholde de kvantitative data. Nedenfor er listen over materialer/komponenter, der anvendes til vores produkt:

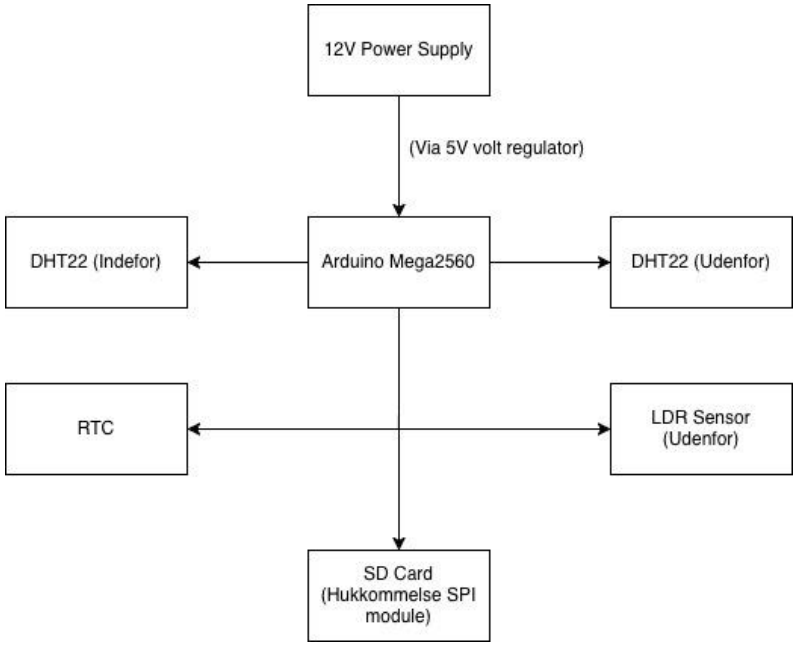
Materialeliste:

Generelt:	Arduino Mega 2560 Breadboard (Stor) 2 DuPont Wires (5v & GND)
Fotomodstand (LDR)	10k Ω Resistor 3 DuPont Wires
2x DHT22	6 DuPont Wires
RTC DS3231	CR2032 Batteri 4 DuPont Wires
Micro SD-Kort *Hukommelse SPI Module*	Micro SD-Kort 4GB 6 DuPont Wires

Block diagrammet er en visualisering af systemet og fungerer som en forenklet grafisk repræsentation af systemets overordnede opbygning. Diagrammet anvender blokke til at

repræsentere de forskellige elementer i systemet samt deres indbyrdes relationer og interaktioner med hinanden.[3]

Figur 2, viser forholdet mellem de enkelte komponenter, der er tilsluttet mikrocontrolleren, som fungerer som systemets centrale styreenhed. Mikrocontrolleren er det centraliserende kontrolcenter, der sikrer, at hver komponent udfører sin specifikke opgave, og gør det muligt for systemet at indsamle den kvantitative data med henblik på at overvåge ændringer i de afhængige variable, herunder fugtighed, temperatur og lysintensitet.[4]



Figur 4.1 Block Diagram

[3] Miro. "What Is a Block Diagram and How to Create One | Miro." *Https://Miro.com/*, miro.com/diagramming/what-is-a-block-diagram/. Accessed 15 Dec. 2025.

[4] GeeksforGeeks. "Microcontroller and Its Types." *GeeksforGeeks*, 20 Feb. 2022, www.geeksforgeeks.org/digital-logic/microcontroller-and-its-types/.

4.2 Software Overview: [navn]

I dette afsnit vil der blive gennemgået selve koden, samt hvilket softwareprogram vi har brugt og hvilket programmeringssprog det skrevet i. Koden er blevet skrevet i softwareprogrammet Arduino IDE (Integrated Development Environment), som er et softwareprogram, der er beregnet til at bruge til Arduino, da dens indbyggede funktioner til at skrive kode, compiler- og upload funktion gør den imødekommende for nyere programmører. Det var derfor vi valgte at bruge softwareprogrammet Arduino IDE til at skrive kode i, samt uploade koden til vores Arduino Board. I softwareprogrammet Arduino IDE bliver der programmet i programmeringssproget c/c++, hvor vi har skrevet valgt at skrive i c++.

Formålet:

Formålet med programmet er at indsamle data til 5 forskellige datapunkter. Datapunkterne er Temperatur indeni i bistanden, Fugtigheden indeni i bistanden, Temperaturen udenfor bistanden, Fugtigheden udenfor bistanden og til sidst lys intensiteten. Dataene skal bruges til se, hvordan eksterne vejrforhold påvirker bi-koloniens udvikling.

Biblioteker:

I starten af vores kode har vi inkluderet de biblioteker, der var essentielle for at programmet kan fungere. (De kan findes i bilaget Software Overview).

Valg af pins:

Da vi arbejder med et Arduino Board, skal man i koden specifikt definere, hvilke pins man benytter, så Arduino Boardet ved, hvilke pins den skal læse signaler fra.

ChipSelect:

ChipSelect (cs) er defineret som en konstant (Const) integer (int). Den er defineret som en konstant for at den ikke kan ændres under programmeringen ved en fejl. Det ikke nødvendigt, men mere en sikkerhed at den ikke bliver at ændre ved et uheld. Når vi bruger et Arduino mega 2560 Board skal man tilslutte til pin 53. Vi benytter pin 53 på Arduino Boardet, fordi vores model har dedikeret SPI [5]pins til SPI-kommunikation mellem Arduino Boardet og SPI-modulet.

[5] Serial Peripheral Interface (SPI) er en synkroniseret dataprotokol, brugt til hurtig kommunikation med en eller flere eksterne enheder over kort distance.

```
const int chipSelect = 53;
```

LightPin:

Lightpinen (Photo resistoren) er defineret som en konstant integer. Lightpinen er tilsluttet til analog pin 0 (A0). Pinen er tilsluttet til analog input, da vi skal bruge det til at aflæse lys intensiteten.

```
const int lightPin = A0;
```

Dht_INDE & Dht_UDE:

Objekterne hedder dht_INDE og dht_UDE, da vi har to sensorer, hvor den ene skal bruges til måle både temperatur og fugtighed indeni, og den anden sensor skal bruges til at måle både temperaturen og fugtigheden udenfor. Disse objekter hører til klassen "DHTNEW". Klassen kommer fra biblioteket "dhtnew.h", hvor klassen er en indbygget i biblioteket, som bruges til temperatur- og fugtighedsmåling, med automatiseret sensor genkendelse. Dht_INDE er tilsluttet til pin 2 på Arduino Boardet, og dht_UDE er tilsluttet til pin 3 på Arduino Boardet.

```
DHTNEW dht_INDE(2);
DHTNEW dht_UDE(3);
```

RTC:

Rtc er et objekt, som tilhører klassen "RTC_DS3231", som er en indbygget klasse i biblioteket "RTClib.h". Biblioteket giver adgang til nuværende tidsværdier, såsom datoen og tidspunktet. Derudover bruger RTC-Modulet I2C[6]protokollen, hvor den bruger to tilslutninger til at sende og modtage data. Ligesom før har Arduino mega 2560 Boardet dedikerede pins til de to tilslutninger, som benytter I2C protokollen, SDA og SCL. En af de dedikerede pins er 20 som er til SDA, som er den som sender data mellem RTC-Modulet og Arduinoen. Den anden er pin 21, som er til SCL, som sørger for at modtage et regulært interval af elektriske impulser fra Arduino Boardet. Disse elektriske impulser bliver brugt til at sørge for at systemet kører præcist og fungere som det skal.

```
RTC_DS3231 rtc;
```

[6] Inter-integrated (I2C) Protocol tillader kommunikation mellem enheder eller sensorer via et to-ledningsinterface kaldet en bus. Bussen består af SDA, som står for at sende og modtage data mellem RTC-Modulet og Arduino Boardet og så SCL, som står for at modtage et regelmæssigt interval af elektriske impulser fra Arduino Boardet. Disse elektriske impulser sørger for at systemet er synkroniseret og kører præcist.

Variabler:

I begyndelsen af koden bliver der dannet nogen variabler, der er vigtige for programmet funktionalitet. Disse variabler opbevarer værdier, og sætter rammen for hvornår den kører.

lastLog:

Variablen ”lastLog” bruges til at gemme det tidspunkt, hvor der sidst blev logget data. Tidsværdien bliver gemt i millisekunder, da det bliver gemt i millisekunder, bliver tallene meget hurtigt ekstrem højde, derfor er variablen defineret som en unsigned long, da den så kan lagre svarende til 32bits. Dette bliver gjort for at risikerer at variablen ikke løber tør for plads, så tidspunktet ikke går tabt.

```
unsigned long lastLog = 0;
```

readCount:

Variablen ”readCount” er en meget simpel integer, der lagrer hvor mange gange der er blevet aflæst data. Selve variablens funktion vil blive gennemgået i loopet.

```
int readCount = 0;
```

Interval:

Variablen ”Interval”, som lægger i navnet, er det programmets interval for, hvornår programmet skal foretage sig en ny aflæsning. Variablen ”Interval” er målt i millisekunder, derfor har den fået værdien 600.000, hvilket svarer til at programmet foretager sig en aflæsning hvert 10minut. Intervallet er sat til hvert 10minut, da det er et passende interval til at aflæse data i. Derudover er variablen også defineret som en unsigned long (32bits), da tidsværdien hurtigt kan overstige en standard variabel som en integer, som kun har 16 bits.

```
unsigned long interval = 600000UL;
```

Setup ():

Setuppet bliver dannet lige før loopet, her bliver der gjort klar til at loopet kan kører uden problemer, ved at opsætte de forskellige pin-tilstande, samt kontroller om initialisering af de forskellige moduler er ordentligt opsat. Derudover kører setuppet kun en gang ved tilføjelse af strøm, hvor loopet derfra tager over.

Initialisering af seriel kommunikation:

Serial.begin er en funktion, der bruges til at initialiserer den serielle kommunikation imellem selve Arduino Boardet og enheden i brug. Derudover benytter den baud rates, som er en seriel dataoverførelse i bits pr sekund. Dette bliver brugt når man skal kommunikere med Serial Monitor, som er et godt redskab til at få printet resultatet af ens kode i terminalen. Vi har brugt det igennem vores programmeringsproces til at gennemtjekke om alt fungerede rigtigt, samt gennemgå om vi fik de ønskede resultater. Herunder har vi sat vores bits pr sekund til 9600, som er meget almindelig når man arbejder med Arduino projekter.

```
Serial.begin(9600);
```

Forhindre forsinkelse ved opstart:

LastLog funktion i setuppet er at den første logning bliver øjeblikkelig startet, efter setuppet er færdigt med at køre, så man ikke skal vente intervallets længde (10min).

```
lastLog = millis() - interval;
```

Delay:

Ved opstart af programmet har vi lavet et lille delay på 2sekunder. Dette er gjort for at programmets systemer har lidt tid til at opstarte, inden de bliver kontrolleret om de er initialiseret korrekt.

```
delay(2000);
```

Opsætning af pin-tilstande:

ChipSelect:

Herunder bliver chipSelect (pin 53) konfigureret som et output. Dette bliver gjort, da chipSelect kommunikere via. SPI-protokollen, og chipSelect skal kunne have kommunikation mellem masteren (Arduino Boardet) og slave (SD-Kortet). Så den bliver sat til output, så den kan skifte mellem low- og high tilstand. Dette gør at den kan blive aktiveret ved low tilstand og deaktiveret ved high tilstand.

```
pinMode(chipSelect, OUTPUT);
```

LightPin:

Herunder bliver lightPinen (pin A0) konfigureret som et input. Dette er gjort så Arduino Boardet ved at den skal modtage data overførelser fra pin A0, ved hjælp af funktionen analogRead, der først bliver taget i brug, i loopet. Derudover er det også gjort, så lightpinen bliver konfigureret til en high tilstand, hvilket gør at den har en ekstrem lille modstand på kredsløbet.

```
pinMode(lightPin, INPUT);
```

Kontrol af SD-Kortets initialisering:

For at kontrollere initialisering af SD-Kortet har vi opsat et enkelt if-statement, der tjekker om chipSelect er aktivt. Hvis den ikke er aktiv, altså at den ikke er initialiseret korrekt giver den en meddelelse i Serial Monitor, at der er fejl med SD-Kortet. Derudover bliver programmet stoppet, i et uendeligt loop, da vi ikke vil miste data hvis der er fejl med SD-Kortet ved opstart.

```
if(!SD.begin(chipSelect)){
  Serial.println("Fejl med SD-Kortet!");
  myFile.close();
  while(1);
}
```

Kontrol af RTC-Modulet initialisering:

Ligesom før benytter vi os af samme koncept, hvor vi kontrollere initialisering af RTC-Modulet, ved brug af et if-statement. Dette if-statement tjekker om RTCen er aktiv, hvis dette ikke er tilfældet, vil den først give en meddelelse om at der er fejl med RTCen til Serial Monitor. Derudover åbner den vores fil “DALO.csv”, hvor vores data blive logget. Her i bliver der logget en fejl meddelelse i vores fil, hvis den ikke er aktiv, samt lukkes filen efter den er blevet logget. Efter bliver der skrevet i Serial Monitor at meddelelsen er logget. Hvis filen ikke kunne blive sendt, vil der blive vist en meddelelse i Serial Monitor at fejlen ikke kunne blive logget.

```
if(!rtc.begin()){
  Serial.println("Fejl med RTCen!");

  myFile = SD.open("DALO.csv", FILE_WRITE);
  if(myFile){
  myFile.println("Fejl med RTCen!");
  myFile.close();
  Serial.println("Fejlen er logget i DALO.csv");
  }else{
    Serial.println("Kunne ikke logge RTC-Fejlen i DALO.csv!");
  }
}
```

Justering af tiden ved opstart:

Herunder benytter vi funktionen rtc.adjust, som er en funktion fra biblioteket "RTClib,h". Funktionen gør at vi ændrer tiden til den nuværende tid, når programmet bliver compilet. Altså at vi manuelt opdaterer tiden for hver gang vi starter programmet. Derfor skal man så også compile, hver gang programmet bliver standset, ellers vil den gå tilbage til den tid der sidst var hentet fra compileren.

```
rtc.adjust(DateTime(__DATE__, __TIME__));
```

Loop ():

Loopet er placeret i slutningen af vores kode. Det her alle funktionerne bliver kørt i et loop, som lægger i navnet, er det her koden bliver kørt igen og igen, dog kun indenfor vores interval på 10min. Denne del af koden er hvor vi aflæser vores data fra sensorerne, samt logger det i vores "DALO.csv" fil. Derudover bliver der også dannet data sortering, af de værdier vi aflæser og logget fra sensorerne.

Tidstrying med millis():

Herunder har vi lavet et if-statement, hvor intervallet bliver taget i effekt. Det her vi benytter funktionen millis() og lastLog til at tjekke hvornår der sidst var logget. Vi bruger dem til at tjekke om lastLog i millisekunder er større end det givne interval. Hvis dette er tilfældet, er der gået de 10min, og så ved programmet at den skal lave en ny logning.

```
if(millis() - lastLog >= interval){
lastLog = millis();
```

Logning tæller:

Den korte kode herunder er til at lægge en til variablen readCount, for hver gang loopet har kørt en omgang, altså hvert 10minut.

```
readCount ++;
```

Datetime:

Herunder benytter vi klassen DateTime fra biblioteket "RTClib.h", hvor vi danner et objekt ved navn "now". Dette objekt funktion er at gøre, så man kan gemme den aktuelle dato og tid fra RTC-Modulet.

```
DateTime now;
```

Kontrollering af RTC-Modulet initialisering i loopet:

Herunder laver vi endnu en kontrol for RTC-Modulet, her er det bare placeret i loopet. Vi har placeret endnu en kontrol i loopet for at tjekke om RTC-Modulet fejler under programmet kørsel. Så ligesom før tjekker den om RTC-Modulet er aktivt, her bruger vi bare en bool variabel, altså en variabel der tjekker om noget er sandt eller falsk. Så if-statement tjekker om RTC-Modulet er aktivt, hvis det er tilfældet, henter den tidspunktet fra objeket "now", hvor hvis den fejler bliver bool variablen sat til sand.

```
  bool rtcFejl = false;

//Tjekker om rtcen fejler.
  if(!rtc.begin()){
    rtcFejl = true;
  }else{
    now = rtc.now();
  }
```

RTC-Modulet fejler:

Hvis bool variablen er sand er det fordi RTC-Modulet fejler. Hvis det er tilfældet, vil koden herunder gøre så programmet forsætter, dog med den bekostning at tiden ikke fungere. Så den vil bare returnere "0000:00:00" for datoen og "00:00:00" for tiden. Herunder i koden hvor der står rtcFejl ? Er bare i stedet for it if-statement, så det en kortere og mere effektiv metode at skrive det på.

```
String date = rtcFejl ? "0000:00:00": now.timestamp(DateTime::TIMESTAMP_DATE);
String time = rtcFejl ? "00:00:00" : now.timestamp(DateTime::TIMESTAMP_TIME);
```

Headeren:

Headeren er overskriften, der bliver brugt til vores "DALO.csv" fil, samt i Serial Monitor. Vi benytter headeren til at skabe struktur, og gøre det nemt for os at se hvad hører til hvad. Vi har implementeret headeren, ved brug af et if-statement. I if-statementet har vi taget variablen readCount i brug, hvor vi bruger den til at tjekke, hvor mange gange der er logget. If-statementet gør at headeren bliver fremvist med det samme, og så skrifter til et interval, hvor den tjekker hver 6 aflæsning. Vi har valgt at lave en header for at gøre det mere overskueligt, samt så headeren kommer frem hver time, så når headeren kommer igen, ved man at der er gået en time.

```
if (readCount == 1 || (readCount - 1) % 6 == 0) {
  Serial.println("| Dato |  Tid | Temp_INDE |Fugt_INDE| Temp_UDE  | Fugt_UDE  | LYS_INTENSITET  |");
```

Herunder i koden åbner den også "DALO.csv" filen, hvor den skriver headeren ind i filen. Den bliver placeret i korrekt orden, så dataene ligger under den rigtige kolonne. De bliver placeret i korrekt orden ved brug af semikolon, da det i en dansk version af en csv fil skrifter kolonne. Derudover bliver filen lukket igen, når den er færdig med at skrive det ind.

```
myFile = SD.open("DATA.csv", FILE_WRITE); //Laver
  if(myFile){
    myFile.print("Dato"); myFile.print(";");
    myFile.print("Tid"); myFile.print(";");
    myFile.print("Temp_INDE"); myFile.print(";");
    myFile.print("Fugt_INDE"); myFile.print(";");
    myFile.print("Temp_UDE"); myFile.print(";");
    myFile.print("Fugt_UDE"); myFile.print(";");
    myFile.println("Lys_INTENSITET");
    myFile.close(); //Lukker filen.
  }
}
```

Sensor aflæsning:

Herunder tager vi objekterne dht_INDE og dht_UDE, i brug ved at benytte funktionen ".read()". Funktionen ". read()" er fra biblioteket "DHTNEW.h", som bruges til at aflæse temperatur og fugtighed både inde og ude.

```
dht_INDE.read();
dht_UDE.read();
```

Opbevaringsvariablerne:

Herunder lagre vi så temperaturen og fugtigheden i de fire variabler, vi har lavet til floats så de også for decimalerne med. Vi anskaffer temperatur og fugtigheden, ved brug af funktionen ". getTeperature()" og funktionen ".getHumidity", som også kommer fra biblioteket "DHTNEW.h".

```
float Temp_INDE = dht_INDE.getTemperature();
float Fugt_INDE = dht_INDE.getHumidity();
float Temp_UDE = dht_UDE.getTemperature();
float Fugt_UDE = dht_UDE.getHumidity();
```

Omdanner værdierne:

Herunder danner vi nogen nye variabler, som skal bruges til at omdanne de givne temperaturer- og fugtighedsmålinger. Vi benytter de her variabler til at omdanne målingerne til heltal, ved at gange det givne målinger med 10. Så ender det med at hvis vi havde en temperatur på 25,5, får vi en værdi på 255. Dette er gjort fordi Arduino ikke har nogen floating point processor, hvilket gør at den bliver langsommere, når den skal behandle float (komma tal).

```
int TEMP_INDE_RESULTAT = (Temp_INDE * 10);
int FUGT_INDE_RESULTAT = (Fugt_INDE * 10);
int TEMP_UDE_RESULTAT = (Temp_UDE * 10);
int FUGT_UDE_RESULTAT = (Fugt_UDE * 10);
```

LDR_Value:

LDR_Value er en normal integer, der bruges til at aflæse lys intensiteten, ved brug af analogRead af lightpinen. Da vi bruger et Arduino mega 2560 Board har den en indbygget funktion analog til digital konverter (ADC), som er essentielt for at man kan aflæse lys intensiteten til læselige værdier. Derfor ved brug af converteren kan den aflæse mængden af spænding lightpinen opfanger og konvertere det til læselige heltal, som går mellem 0 – 1024. Disse værdier som sagt går mellem 0 – 1024, hvor 0V er 0 og 5V er 1023 og ikke 1024, da man også tæller 0 som en værdi.

```
int LDR_Value = analogRead(lightPin);
```

Sorting af Data:

Som vores metode til at sortere data, altså vores temperatur og fugtighedsmålinger. Har vi valgt at lave et if-statement, der tjekker om der er fejl med målingen. Hvis der er fejl med målingen, returnerer den –999. Dette betyder at der er opstået en fejl under afmåling. Derfor har vi valgt at lave et if-statement der sorterer fejl resultaterne ud, så de ikke bliver skrevet ind på Serial

Monitor eller bliver logget i “DALO.csv” filen. Vi tager igen brug af print funktionen til at skrive det ind i både Serial Monitor og vores fil, hvor vi åbner og lukker filen efter brug.

```
if(Temp_INDE == -999 || Fugt_INDE == -999 || Temp_UDE == -999 || Fugt_UDE == -999){

  }
   else{

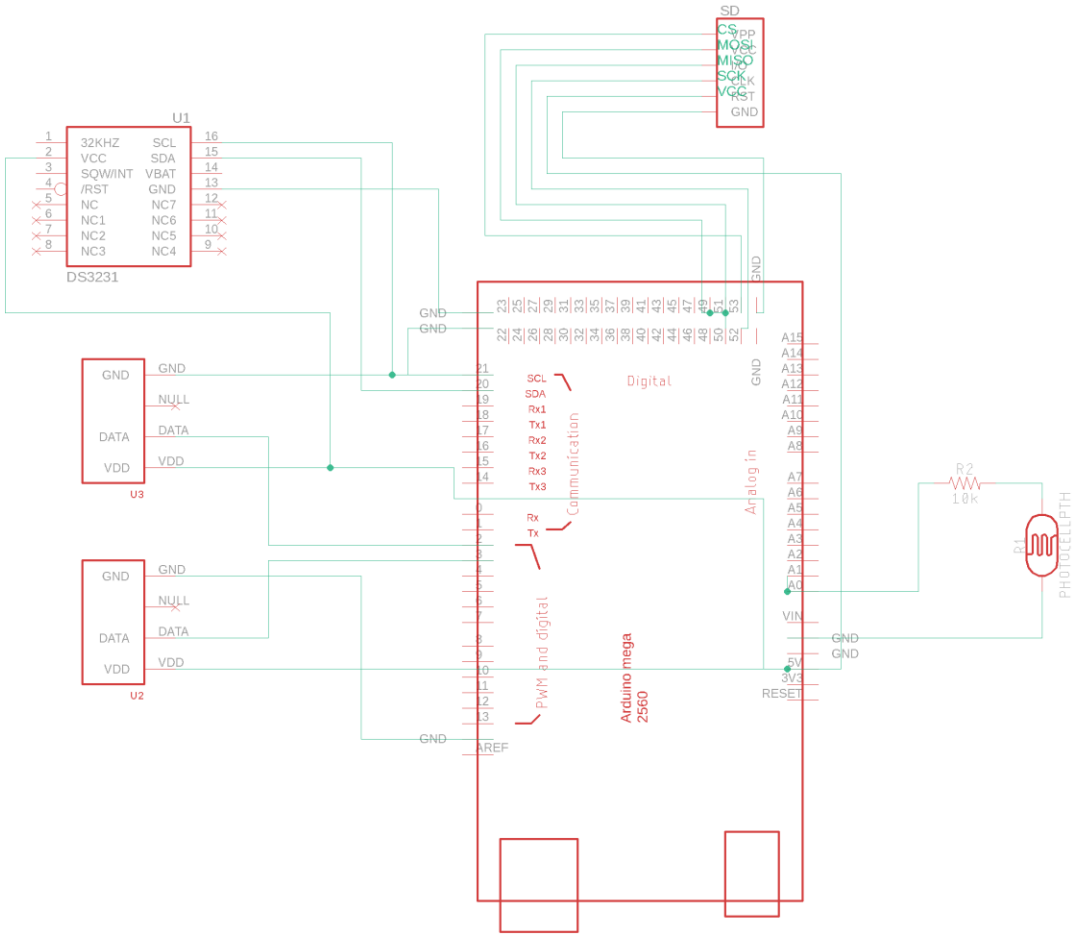
  Serial.print(date); Serial.print(",");
  Serial.print(time); Serial.print(",");
  Serial.print(TEMP_INDE_RESULTAT); Serial.print("C,");
  Serial.print(FUGT_INDE_RESULTAT); Serial.print("%,");
  Serial.print(TEMP_UDE_RESULTAT); Serial.print("C,");
  Serial.print(FUGT_UDE_RESULTAT) ; Serial.print("%,");
  Serial.println(LDR_Value);


  myFile = SD.open("DALO.csv",FILE_WRITE);
  if(myFile){
    myFile.print(date); myFile.print(";");
    myFile.print(time); myFile.print(";");
    myFile.print(TEMP_INDE_RESULTAT); myFile.print("C;");
    myFile.print(FUGT_INDE_RESULTAT); myFile.print("%;");
    myFile.print(TEMP_UDE_RESULTAT); myFile.print("C;");
    myFile.print(FUGT_UDE_RESULTAT); myFile.print("%;");
    myFile.println(LDR_Value);
    myFile.close();
    }
  }
}
```

4.3 System Architecture: [navn]

Denne sektion af systemoversigten fokuserer på systemarkitekturen, som beskriver hvordan systemets hardware og software kommunikerer med hinanden som en enhedsløsning. Mikrokontrolleren fungerer som en central kontrolenhed, der modtager data fra de tilsluttede komponenter, herunder DHT og LDR-sensorerne. RTC-modulet håndterer tidsstempling og gemmer de kvantitative data i mikro SD SPI-modulet. Figur 3, skemaer, viser alle de elektriske forbindelsers indbyrdes forhold, der gør det muligt for den centrale kontrolenhed at kommunikere med og modtage information fra de tilsluttede komponenter; det er broen mellem hardware og software, der gør det muligt for systemet at opnå sit mål om at måle og overvåge bistadet. Der er flere kommunikationsmetoder, som systemet bruger: de første DHT-sensorer kommunikerer via digitale I/O-porte, LDR’en læses gennem en analog indgang, RTC-modulet bruger I²C-kommunikation, og endelig bruger SD-kortet SPI-kommunikation. Forskellen mellem

skemaerne (Figur 3) og blokdiagrammet (Figur 2) er, at skemaerne viser de detaljerede og præcise elektriske forbindelser, mens blokdiagrammet repræsenterer en top-ned oversigt, der viser forholdet mellem dem.



Figur 4.3 - Skematiks

5. Analyse:

I dette afsnit gennemgår vi først analysen af vores data og sammenligner denne med andre gruppers. Efterfølgende vurderer vi vores produkts kvalitet og hvilke fejlkilder vi har fundet.

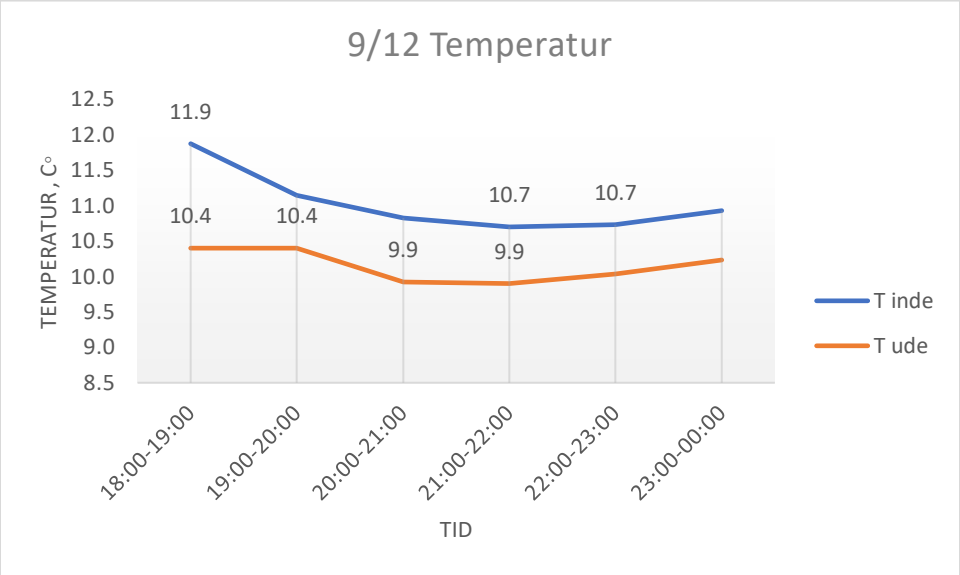
Dataanalyse: Lea

For at få et overblik over temperatur, luftfugtighed og lysintensitet, tager vi vores data og finder middelværdierne time for time. Vi beregner middelværdien med formlen $\frac{Sum\ af\ målinger}{Antal\ måling}$, som sættes i en graf for nem aflæsning.

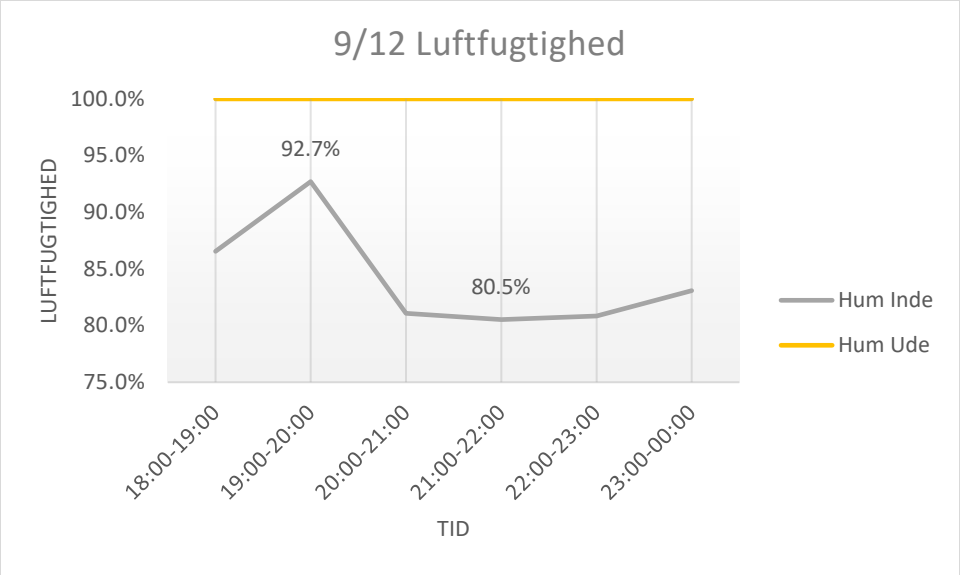
Følgende er grafer for de individuelle dage, og efterfølgende en graf af udviklingen over alle dage og kommenterer på lysintensiteten. Vores datasæt kan tilgås i bilag E.

9. December:

Vores datalogger er sat til d. 9. december omkring klokken 18. Den højeste middelværdi for temperaturen er 11,9 C° inde i kassen og 10,4 udenfor. Luftfugtigheden uden for kassen er på 100 %, så det regner. Fugtigheden indenfor er 92,7 % på sit højeste og det mindste er 80,5 %. Da solen er gået ned og kassen står væk fra en lyskilde, er der ingen data fra lyssensoren.



Figur 5.1 - Temperatur

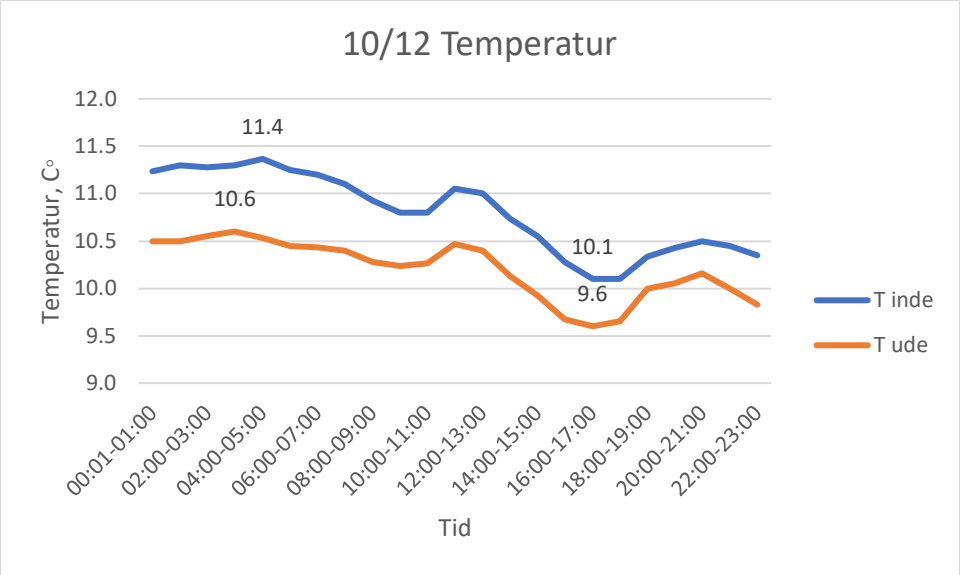


Figur 5.2 - Luftfugtighed

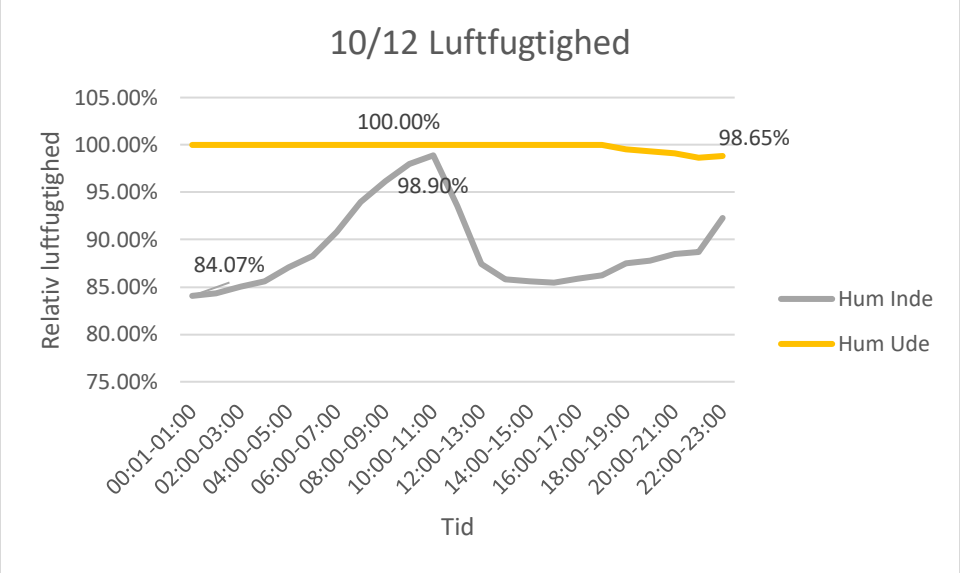
10. december

Denne dag bød på en fortsættelse af tendensen fra tidligere, temperaturen inde i 'bikuben' er højest af de to, og ligger på 11,4 C og 10,1 C i sit laveste. Uden for 'bikuben' er det højeste 10,6 C og det laveste er på 9,6. Luftfugtigheden udenfor tyder stadig på regn, og inde i bikuben er det fortsat lavere luftfugtighed i forhold til udenfor. Der sker en stigning i luftfugtighed i løbet af dagen. Det håndklæde vi har lagt mellem vores to kasser er vådt, så det er højst sandsynligt grunden til denne måling.

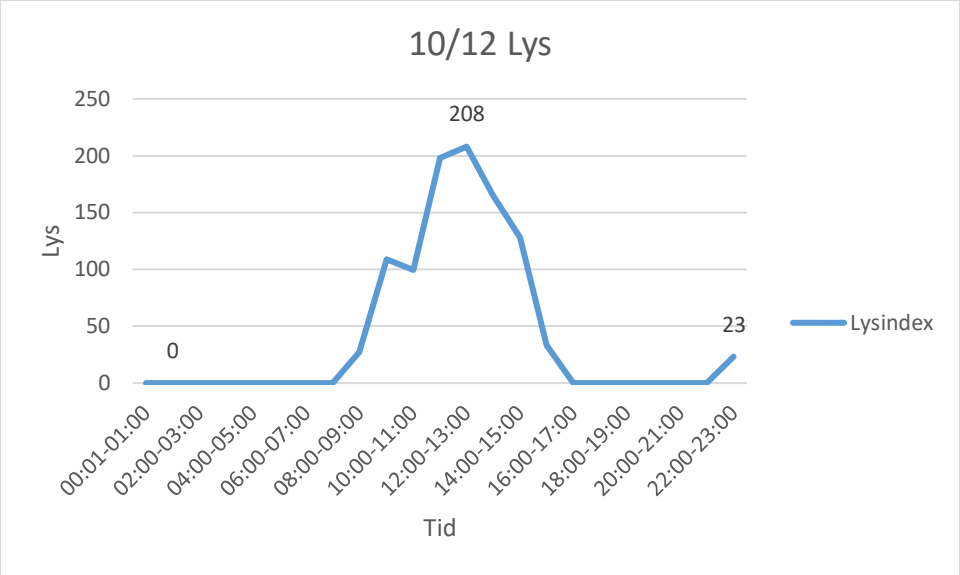
Vores lyssensor har haft chancen for at opfange lys, og fra omkring 8.00 til 18.00, har data. Dette passer meget godt sammen med, at solen er stået op. Klokken 22-23 får vi dog en høj måling på lyssensoren på 23,25. Dette er fordi der er brugt en lommelygte for at tjekket dataloggeren. Ser man bort fra dette, har sensoren registreret lys fra klokken cirka 8.30 til 15.30, det er 7 timer med nok lys til at blive registreret. Det ville næppe opfattes som solskin. Denne dag havde dataloggeren ingen data fra klokken 23-00, vi er usikre på hvorfor.



Figur 5.3 - Temperatur



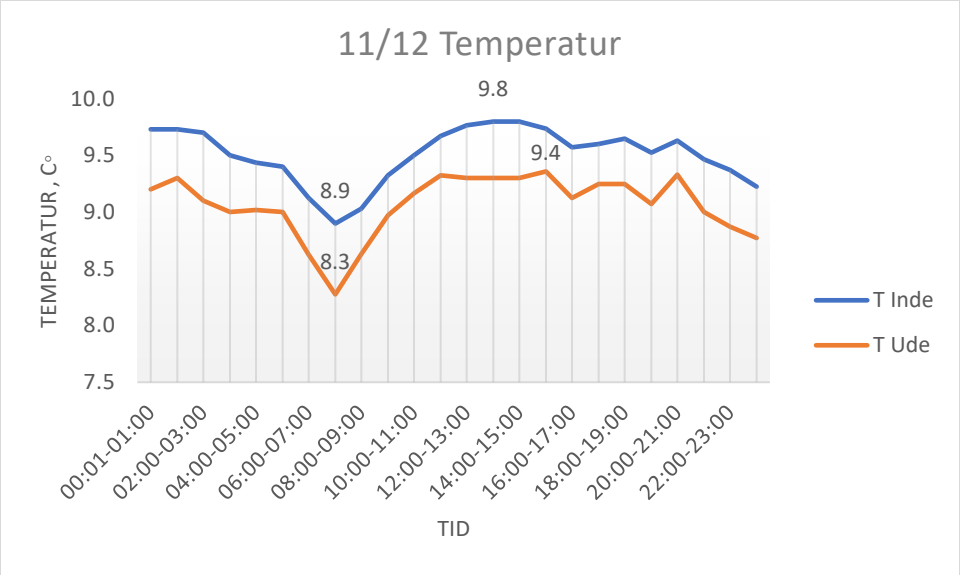
Figur 5.4 - Luftfugtighed



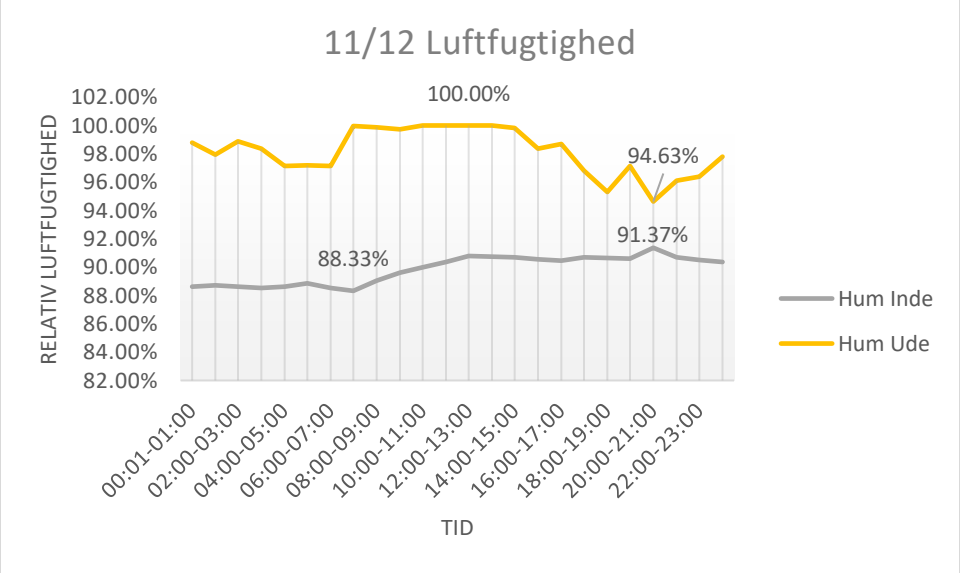
Figur 5.5 - Lys Intensitet

11. december

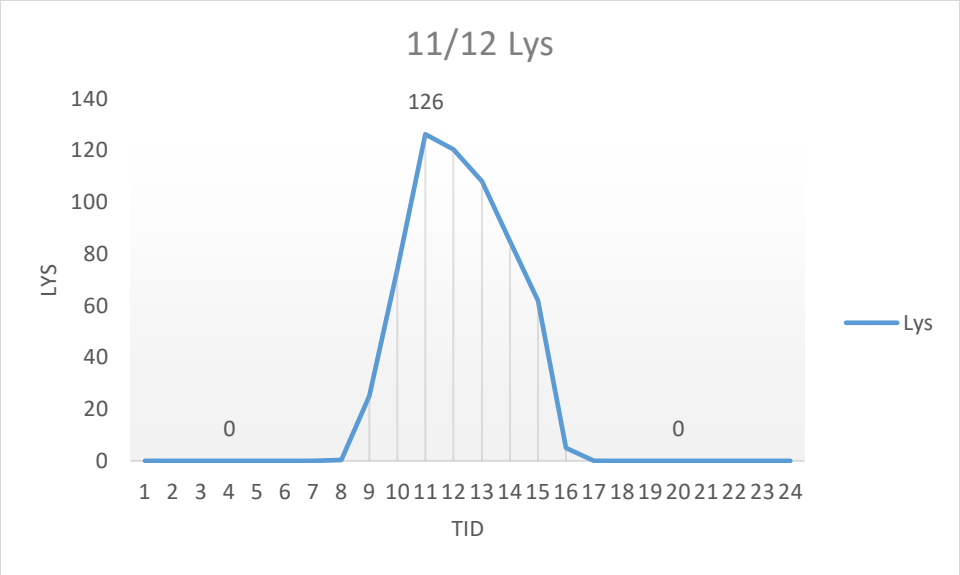
Data fra denne dag følger den samme tendens som de foregående. Temperaturen inden i kassen er mellem 9,8 og 8,9 C, hvor den udenfor er mellem 9,4 og 8,3 C. Fugtigheden udenfor kassen lægger stadig op til regn det meste af tiden. Vi har ikke længere en høj luftfugtighed i løbet af dagen, vi tænker at dette kan være fordi vi har skiftet håndklædet. Lyssensoren registrer lys i dagstiden, der har været lys cirka fra klokken 8.30 til 15.50. Det er 7 timer og 20 minutter med lys, men på intet tidspunkt er det direkte solskin.



Figur 5.6 - Temperatur



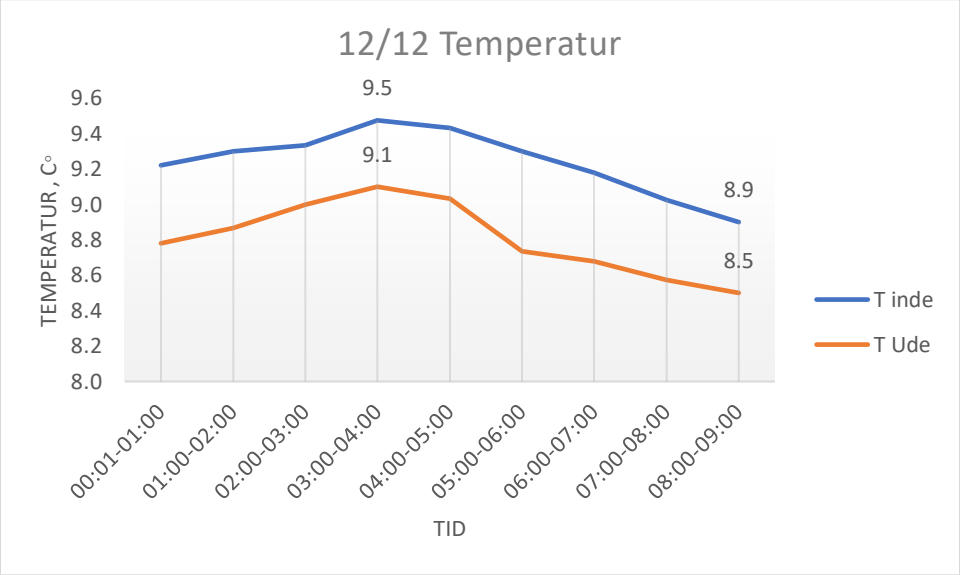
Figur 5.7 - Luftfugtighed



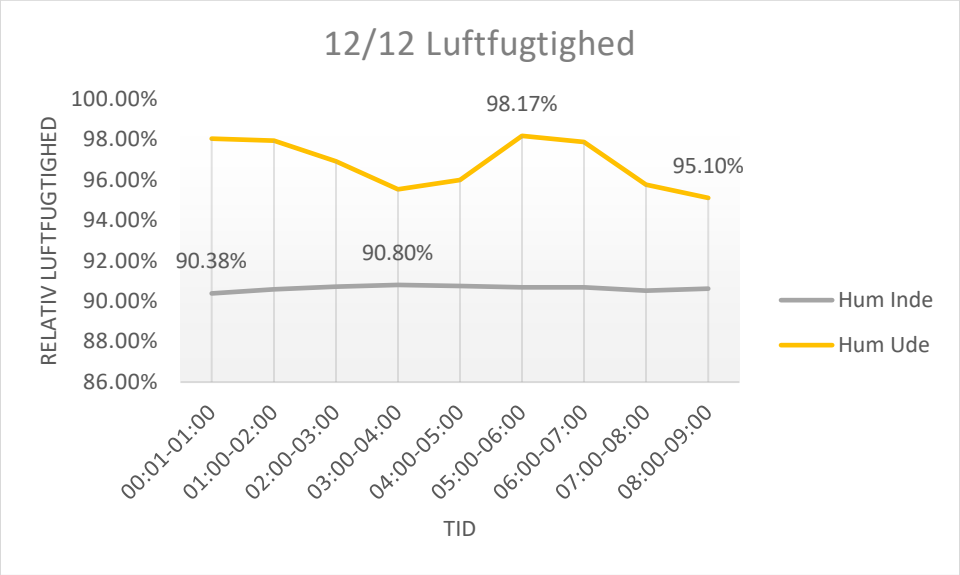
Figur 5.8 - Lys Intensitet

12 December

Dataloggeren bliver taget ned klokken 9, derfor er der ikke mere data efter dette tidspunkt. Der er en maks temperatur på 9,48 indenfor og 9,10 C udenfor. Minimum ligger på 8,9 indenfor og 8,5 udenfor. Temperaturerne indeni og udenfor kassen ligger meget tæt på hinanden. Luftfugtigheden ligger stabilt på de cirka 90 procent. Der er ingen data på lys, da solen endnu ikke er stået op.



Figur 5.9 - Temperatur

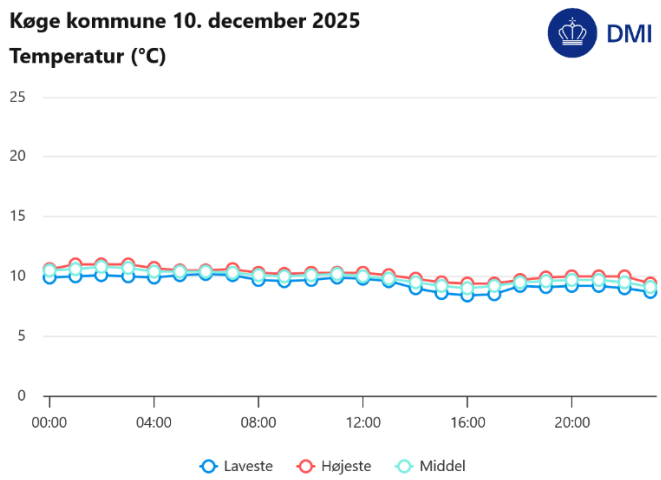


Figur 5.10 - Luftfugtighed

Samlet analyse:

Vores dataloggers inde-temperatur ligger generelt omkring 1 C over temperaturen udenfor, og følges pænt ad. Den relative fugtighed er generelt meget høj udenfor, og der har været meget regn og overskyet. Indenfor er luftfugtigheden stadig høj og ligger højt. Ser man bort fra den høje værdi om morgen den 10/12, som skyldes en fejlkilde, er luftfugtigheden rimelig stabil på omkring de 90 % inden i bikuben.

Sammenligner vi vores data med vejrudsigten fra Danmarks Meterologiske Institut, DMI, ligger vores temperaturmåling på linje med vejrudsigten for perioden:



Figur 5.11 – Temperaturen for d. 10 december, 2025 i Køge. Den ligger på linje med vores målinger.

Vores temperatur målinger ligger tæt på DMI's målinger for temperatur d. 10/12. Det giver os et hurtigt indblik i at vores datalogger måler og noterer de rigtige temperaturer.

Overordnet set får vi de rigtige målinger, men vi får ikke alle de datapunkter, vi kunne have ønsket. Med vores opsætning burde dataloggeren give seks målinger per time, i 63 timer. Dette er 378 punkter. Vi har 214 datapunkter, det giver en logging-effektivitet på $\frac{214}{378} * 100 = 56{,}6\ \%$.

Vi har altså et data tab på 43,4 %.

Produkt analyse: Lea

Vores datalogger kan måle temperatur, fugtighed og lysintensitet som ønsket. Den data gemmes i en csv fil på SD-kortet. Data er organiseret i de rette kolonner, og det er nemt at aflæse.
I løbet af den tid hvor dataloggeren har været sat up, har den dog kun registreret data 56 % af tiden. Der er plads til forbedring.

Papkassen har holdt vand og fugt væk fra vores microcontroller, og kredsløbet har siddet korrekt sammen under hele forløbet.

LDR-resistoren måler lavere end vi havde regnet med, og vi havde gerne set nogle højere værdier. Det har været svært for os at bedømme om den målte rigtigt eller ej. Dataloggeren er sat op om vinteren, og der har været meget regn og overskyet det meste af tiden. Den generelle tendens passer, vi kan se at solen står op og går ned igen.

Simulering af bikube.

For at have mulighed for at tjekke op på vores datalogger, har vi simuleret en bikube ved hjælp af to kasser.

Vi valgte at oprette vores data i en csv fil, og fik opsat et pænt og letlæseligt skema. Vi skulle dog have sørget for at der ikke var enheder i vores dataset. Det var let at rette til i Excel, men en irriterende og åbenlys ting at komme udenom i koden.

Koden kunne sikkert være sat op så den selv regnede middelværdien, men dette kom vi først i tanke om efter dataindsamling. Dette havde sparet tid i dataanalysen.

Fejlkilder: [navn]

Kvaliteten af komponenterne:

Under vores dataindsamling observerede vi at der var mange fejl målinger, hvilket medførte at vores softwaresystem sortererede dem fra. Det resulterede i et lavere antal brugbare datasæt end vi havde forventet. Dette skete også under testene før dataindsamlingerne, hvilket indikerer at selve sensorerne ikke var af bedste kvalitet. Da de allerede fejlede under gode vilkår, inden de blev placeret udenfor, hvor fugt kunne være en faktor.

Photo resistor:

Under dataanalysen fandt vi ud af at der var et fejl med afmålingerne fra vores Photo resistor. Vores Photo resistor målinger var meget ved siden af det man kunne forvente. Dette skyldes at der var en fejl i opsætningen eller at vores Photo resistor var defekt. Man kunne nemt se ved databehandlingen at der var noget galt med vores lys intensitet afmålinger, da vores højeste afmåling var 207 og laveste 0. Ud fra disse afmålinger kan vi konkludere at der er opstået en fejl i processen.

Simulering af bikube:

Vores simulering af en bikube er ikke særlig præcis, da den er lavet ud fra det tilgængelige ressourcer i eget hjem. Den var ikke lavet til at modstå det dårlige vejr, vi havde første dag, vi indsamlede data. Dette kunne påvirke vores fugtigheds-, samt temperaturdata, da der var ingen form for indbygget isolation, som kunne håndtere, den mængde fugt. Derudover var der et håndklæde, der blev placeret imellem trækasserne i forsøg for at beskytte den indre kasse for alt for meget fugt. Hvor der er stor chance for at den kunne havde haft den modsatte effekt, da det dårlige vejr hurtigt gjorde håndklædet fugtigt. Håndklædet blev skiftet næste dag, men dataindsamlingen på første dag kunne havde blevet påvirket af den grund.

6. Konklusion: Lea

Ved hjælp af komponenter til en microcontroller har vi designet og bygget et indlejret system til at måle de miljøforhold, som er relevante for biers overlevelse. Denne data kan bruges til at lave tiltag som sikrer biers levevilkår.

Vi har brugt SCRUM til at sætte rammerne for processen og produktudvikling. Det gav grundlæggende overblik over forløbet, og idéer til løsningen af del-opgaver i udvikler-team. Den kontinuerlige test af vores produkt har resulteret i udfordringer hurtigt er blevet løst. Metoden hjalp også til at holde overblik over tiden, og være realistiske omkring produktets omfang og kvalitet.

Muligheden for at lave en datalogger med en microcontroller og billige komponenter, gør det muligt for fagfolk såvel som interesseret lægmænd at lave dataloggere. Med muligheden for selv at designe dataloggere, kan mange miljøer nu undersøges og belyses.

7. Videreudvikling:

Koden: [navn]

Automatiseret opdatering af tiden:

I vores overstående kode del, der omhandler opdatering af tiden. Har vi brugt funktionen ”rtc.adjust” fra biblioteket (RTClib.h), der gjorde det muligt at manuelt tilpasse tiden, for hver gang koden blev kørt. Funktionen fungerer på den måde at den henter tiden fra ens enhed, og benytter den til koden bliver kørt igen, hvor den så tilpasser sig igen. Dette er en meget simpel måde at gøre det på, hvor man måske kunne havde lavet en mere effektiv måde, hvorpå man kunne lave en automatiseret opdatering af tid, så man ikke hele tiden skulle manuelt køre filen, så ens tidspunkter ikke går tilbage til, hvornår man sidst kørte den.

Beregning & fjernelse af enheder:

Derudover kunne en forbedring af koden, være at man lavede en kode, der er i stand til at beregne middelværdien. Vi endte i sidste ende ikke med at gøre dette, da vi først efter dataindsamling kom i tanke om muligheden. Derfor endte vi med at have oprettet vores data i en .csv fil, som blev opsat i et struktureret og læseligt skema som ses på bilag E. Dog ved analysen af dataene, fandt vi ud af at tilføjelsen af enhed gav nogen problemer, men ikke noget man ikke kunne rette til i Excel. Det ville være en nem rettelse i koden, som vil forbedre læsningen af data og derfor forbedre koden til at give det bedste resultat.

Fysiske: Lea

Wireless download:

Hvis man vil minimere potentielle bi-stik, kunne man tilføje komponenter så man kan download sin data. Bluetooth-modulet *HC-06* ville give mulighed for at tilgå sin data i op til 10 meters afstand. Der findes også forskellige WiFi-komponenter, som kan give enorm distance. Man behøver teknisk set ikke et SD-kortet, hvis man tilføjer disse komponenter, men det er en fordel at have det som back-up. Dette gør også at man ikke lukker varmen ud fra bikuben, hver gang man tjekker sin sensor.

Komponenter:

De komponenter der er brugt i dette system, er billige og lettilgængelige. De er gode til prototyper og små projekter. Hvis man ønsker mere præcis data, f.eks. til et forskningsprojekt,

bør man vælge komponenter af en højere kvalitet. DHT-22 har en fejlmargin på 0.5 C, så den kunne erstattes af SHT-31, som har en fejlmargin på 0.2 C. Det koster dog flere penge.

Lodning, THT:

Hvis man ønsker at bruge systemet i længere tid, ville det være godt at lodde det sammen for at sikre stabiliteten. Dette kunne gøres med through-hole-technology, også kendt som en THT plade.

Forbedring af kasse:

For at beskytte vores produkt, lavede vi en papkasse og beskyttede bunden med gaffatape. Sensorerne var også fikseret med tape. Dette setup er fint nok til at opsamle data i tre dage, men ikke langtidsholdbart. Ønsker man data over længere tid, f.eks. en sæson eller et år, er denne kasse ikke optimal.

En bedre kasse kan designes og produceres med 3D print. Man kan finde en gratis model som passer til et Arduino megaboard, som ville være en opgradering i beskyttelse af megaboardet. Det er også muligt at designe sin egen boks og give muligheder for god kabelskik. Her kunne man også lave plads til komponenterne, så de er bedre fikseret.

8. Evaluering af samarbejde: Alle

Vores kommunikation har været god, og vi har ikke haft konflikter med hinanden. Vi har været rummelige i tilgangen til fremmøde og sygdom. Alle har løbende meldt ud, hvis der opstår spørgsmål og problemer med opgaven.

Det har været en udfordring at følge SCRUM stringent, men det har holdt vores overordnet ramme og givet en forståelse for hvad der foregår i gruppen.

Opgavefordelingen har lagt vægt på vores styrker, så alle har bidraget til rapporten og produktet.

Alt i alt er vi tilfredse med vores samarbejde i forløbet.

Kildeliste: [navn]

"[RTC] [SOLVED] How to Set Date and Time Permanent." *Arduino Forum*, 13 Sept. 2013, forum.arduino.cc/t/rtc-solved-how-to-set-date-and-time-permanent/183180. Accessed 8 Dec. 2025.

AG, Sensirion. "SHT31-DIS-B-±2% (0-100%RH) Digital Humidity and Temperature Sensor." *Sensirion AG*, sensirion.com/products/catalog/SHT31-DIS-B.

Aljundi, Liam. "Using the Arduino Software (IDE) | Arduino Documentation." *Docs.arduino.cc*, 16 Jan. 2024, docs.arduino.cc/learn/starting-guide/the-arduino-software-ide/.

Amri, Mohd, et al. "Internet of Things (IoT) Application in Meliponiculture." *International Journal of Integrated Engineering -Special Issue on Electrical Electronic Engineering*, vol. 9, no. 4, 2017, pp. 57–63. Accessed 4 Dec. 2025.

Ardustore. https://ardustore.dk/produkt/bluetooth-2-0-wireless-transeiver-hc-06-module. Accessed 15 dec. 2025

Blaszczyk, Tomasz. "Introduction to Embedded Systems." Sept. 2025.

Microsoft Copilot. "Microsoft Copilot." *Microsoft Copilot*, Microsoft, 2025, copilot.microsoft.com/.

Derrick, Scott. "The Importance of Beehives in Nature for Our Ecosystems and Biodiversity." *Blythewood Bee Company*, 23 Mar. 2025, blythewoodbeecompany.com/blogs/news/beehive-in-nature. Accessed 4 Dec. 2025.

"Digital Pins." *Arduino.cc*, 2024, docs.arduino.cc/learn/microcontrollers/digital-pins/.

DMI (Danmarks Meteorologiske Institut). "Danmarks Meterologiske Institut. Lokationsarkiv ." *Www.dmi.dk*, Danmarks Meteorologiske Institut, www.dmi.dk/lokationarkiv/show/DK/2618415/K%C3%B8ge/#arkiv. Accessed 11 Dec. 2025.

Duan, Xiaodong, et al. "ApisRAM Formal Model Description." *EFSA Supporting Publications*, vol. 19, no. 2, Feb. 2022, https://doi.org/10.2903/sp.efsa.2022.en-7184.

GeeksforGeeks. "Microcontroller and Its Types." *GeeksforGeeks*, 20 Feb. 2022, www.geeksforgeeks.org/digital-logic/microcontroller-and-its-types/.

Margolis, Michael. *Arduino Cookbook.* 3rd Edition ed., O'reilly Media, Incorporated, Apr. 2020.

Miro. “What Is a Block Diagram and How to Create One | Miro.” *Https://Miro.com/*, miro.com/diagramming/what-is-a-block-diagram/. Accessed 15 Dec. 2025.

Niels Vestergaard Olsen, and Susanne Muusmann Lassen. *Projektledelse*. 2025. Kbh., Trojka, 2019.

OpenAI. “ChatGPT.” *Chatgpt.com*, OpenAI, 12 Sept. 2025, chatgpt.com.

Schwaber, Ken, and Jeff Sutherland. *Scrum Guiden™ Den Ultimative Guide Til Scrum: Spillets Regler*. scrumguides.org, Oct. 2011.

Scrum.org. “What Is a Sprint Retrospective?” *Scrum.org*, 2019, www.scrum.org/resources/what-is-a-sprint-retrospective. Accessed 11 Dec. 2025.

sparkfun. “GitHub - Sparkfun/SparkFun-Eagle-Libraries: SparkFun’s Public Eagle PCB Footprints.” *GitHub*, 2025, github.com/sparkfun/SparkFun-Eagle-Libraries. Accessed 9 Dec. 2025.

“Wat Is Een SD-Kaart (Secure Digital)? | Lenovo Denmark.” *Lenovo.com*, 2021, www.lenovo.com/dk/da/glossary/sd-cards/?srsltid=AfmBOoouVaqhycTt0pkR7oTZEAtfeEo3qKhgheV360UV6PuipPfF20ct. Accessed 16 Dec. 2025.

“What Is the Operating Temperature Range for Arduino Boards?” *Arduino Help Center*, support.arduino.cc/hc/en-us/articles/360016076980-What-is-the-operating-temperature-range-for-Arduino-boards.

Wokwi. “Wokwi - Online Arduino and ESP32 Simulator.” *Wokwi.com*, wokwi.com/.

Zambetti, Nicholas, et al. “Inter-Integrated Circuit (I2C) Protocol.” *Arduino.cc*, 2024, docs.arduino.cc/learn/communication/wire/.

Arduino.cc, 2024, docs.arduino.cc/language-reference/en/functions/analog-io/analogRead/.

Arduino.cc, 2024, docs.arduino.cc/libraries/sd/.

Arduino.cc, 2024, docs.arduino.cc/learn/communication/spi/.

Arduino.cc, 2025, docs.arduino.cc/libraries/dhtnew/. Accessed 12 Dec. 2025.

Arduino.cc, 2024, docs.arduino.cc/language-reference/en/functions/communication/Serial/begin/.

Kildeliste til kode:

(Blaszczyk) Lab 10, lab 20, Lab 27, lab 28 & lab 29.

Brugt https://chatgpt.com/ til problemløsning.

Bilag:

A. Sprint Backlog:

Sprint backlog er fjernet i den offentlige version.

https://beehiveprokect.atlassian.net/sr/jira.issueviews:searchrequest-prin...

T	Key	Summary	Assignee	Reporter	P	Status	Resolution	Created	Updated	Due
	BHP-14	DEADLINE 17TH OF DECEMBER	Unassigned	Christian Mortensen	=	TO DO	Unresolved	03/ Dec/25	03/ Dec/25	17/ Dec/25
	BHP-13	Schematics	Christian Mortensen	Christian Mortensen	=	DONE	Done	03/ Dec/25	08/ Dec/25	
	BHP-12	Program/code	Christoffer Robert Knudsen	Christian Mortensen	=	DONE	Done	03/ Dec/25	05/ Dec/25	
	BHP-11	5 Sensors in total, 2 Humidity sensors, 2 Temperature sensors, 1 photo resistor.	Christoffer Robert Knudsen	Christian Mortensen	=	DONE	Done	03/ Dec/25	07/ Dec/25	
	BHP-10	Bilag (kun bilag, som er centrale for rapporten.)	Unassigned	Christian Mortensen	=	IN PROGRESS	Unresolved	03/ Dec/25	15/ Dec/25	
	BHP-9	Kildeliste over alle kilder brugt i projektet	Christian Mortensen	Christian Mortensen	=	IN PROGRESS	Unresolved	03/ Dec/25	15/ Dec/25	
	BHP-8	Konklusion	Lea Mette Jacobsen	Christian Mortensen	=	IN PROGRESS	Unresolved	03/ Dec/25	16/ Dec/25	
	BHP-7	Data Analyse	Lea Mette Jacobsen	Christian Mortensen	=	DONE	Done	03/ Dec/25	15/ Dec/25	
	BHP-6	IoT metodeafsnit	Lea Mette Jacobsen	Christian Mortensen	=	DONE	Done	03/ Dec/25	11/ Dec/25	10/ Dec/25
	BHP-5	Indledning inkl. Problemformulering	Christian Mortensen	Christian Mortensen	=	DONE	Done	03/ Dec/25	08/ Dec/25	
	BHP-4	Indholdsfortegnelse (her skal fremgå, hvilke studerende, der har udformet de enkelte afsnit)	Unassigned	Christian Mortensen	=	DONE	Done	03/ Dec/25	04/ Dec/25	
	BHP-3	Forside med titel, navne på deltagere samt navn på vejleder.	Unassigned	Christian Mortensen	=	DONE	Done	03/ Dec/25	04/ Dec/25	

B. Gruppekontrakt:

Gruppekontrakt er fjernet i den offentlige version.

Gruppekontrakt er fjernet i den offentlige version.

Gruppekontrakt er fjernet i den offentlige version.

C. Kildekode (.ino):

Insert your code:

```
//Inkluderer dhtnew.h biblioteket, som bruges til afmålning af Temperatur og Fugtighed.
#include <dhtnew.h>

//Inkluderer Wire.h biblioteket, som bruges til kommunikation med RTC.
#include <Wire.h>

//Inkluderer RTClib.h, som bruges til at hente og danne Timestamps.
#include <RTClib.h>

//Inkluderer SD.h & SPI.h biblioteket for at SD-Kortet og Arduinoen kan snakke sammen.
#include <SD.h>
#include <SPI.h>
```

```
//Definerer chipSelect (CS) til at være pin 53.
const int chipSelect = 53;

//Definerer at man kommer til at arbejde med en fil, og deklarer den specifikke myFile.
File myFile;

//Definerer Photoresistorens (LDR) pin;
const int lightPin = A0;

//Tildeler digitale pins til de to DHT Temperatur- og Fugtighedssensorer.
DHTNEW dht_INDE(2);
DHTNEW dht_UDE(3);

//Opretter et RTC-objekt til DS3231, så Arduinoen kan aflæse dato & tid.
RTC_DS3231 rtc;

//Variabler.
unsigned long lastLog = 0; //Variablen gemmer tidspunktet for sidste loging (ms).
int readCount = 0; //Variablen tæller antal aflæsninger.
unsigned long interval = 600000UL; //Aflæser Temperaturen & Fugtigheden hver 10min.

void setup(){
  Serial.begin(9600);

  lastLog = millis() - interval; //Gør at den ikke kører før intervallet er færdigt.

  pinMode(chipSelect, OUTPUT); //Sætter chipSelect til at være output.

//Start delay på 5sek for at give Arduinoen og Serial Monitor tid til at starte op.
  'delay(2000);

//Definerer Photoresistorens pin som Input.
  pinMode(lightPin, INPUT);

//Tjekker om SD-Kortet er tilsluttet korrekt.
//Stopper programmet i et uendeligt loop, så programmet ikke kører uden SD-Kortet for ikke at midste
data.
  if(!SD.begin(chipSelect)){
    Serial.println("Fejl med SD-Kortet!");
    myFile.close();
    while(1);
  }

//Tjekker om RTCen (DS3231) er tilsluttet korrekt.
//Stopper programmet i et loop, så programmet ikke kører uden RTCen.
  if(!rtc.begin()){
```

```
    Serial.println("Fejl med RTCen!");

//Logger at der er fejl med RTCen i filen DALO.csv.
    myFile = SD.open("DALO.csv", FILE_WRITE);
    if(myFile){
    myFile.println("Fejl med RTCen!");
    myFile.close();
    Serial.println("Fejlen er logget i DALO.csv");
    }else{
      Serial.println("Kunne ikke logge RTC-Fejlen i DALO.csv!");
    }
  }
   //Sætter RTCen til den aktuelle tid ved opstart af programmet.
    rtc.adjust(DateTime(__DATE__, __TIME__));

}

void loop(){
//Styrer intervallet mellem aflæsninger.
  if(millis() - lastLog >= interval){
    lastLog = millis(); // Opdaterer tidspunktet for seneste aflæsning.
    readCount ++; //Lægger 1 til variablen hver gang, der bliver aflæst.

    DateTime now;

  //Opsætter en variabel, der tjekker om rtcen fejler imens programmet kører.
    bool rtcFejl = false;

  //Tjekker om rtcen fejler.
    if(!rtc.begin()){
      rtcFejl = true;
    }else{
      now = rtc.now();
    }

  //Printer Headeren i starten af aflæsningen, derefter hver 6ede aflæsning, altså hver time.
    if (readCount == 1 || (readCount - 1) % 6 == 0) {
      Serial.println("| Dato |  Tid | Temp_INDE |Fugt_INDE| Temp_UDE  | Fugt_UDE  |
LYS_INTENSITET  |"); //Headeren.

  //Tjekker om filen er åben, hvis det er tilfældet vil den skrive headeren.
    myFile = SD.open("DALO.csv", FILE_WRITE); //Laver eller tilgår filen "DALO.csv" og skriver i
den.
      if(myFile){
        myFile.print("Dato"); myFile.print(";");
        myFile.print("Tid"); myFile.print(";");
```

```
      myFile.print("Temp_INDE"); myFile.print(";");
      myFile.print("Fugt_INDE"); myFile.print(";");
      myFile.print("Temp_UDE"); myFile.print(";");
      myFile.print("Fugt_UDE"); myFile.print(";");
      myFile.println("Lys_INTENSITET");
      myFile.close(); //Lukker filen.
    }
   }

  //Aflæser Temperatur og Fugtighed fra både Inde- og Udesensorerne.
   dht_INDE.read();
   dht_UDE.read();

  //Gemmer de aflæste Temperatur- og Fugtighedsværdier fra Inde- og Udesensorerne, som float for at
få decimaler på.
   float Temp_INDE = dht_INDE.getTemperature();
   float Fugt_INDE = dht_INDE.getHumidity();
   float Temp_UDE = dht_UDE.getTemperature();
   float Fugt_UDE = dht_UDE.getHumidity();

  //Omskriver den aflæste Temperatur- og Fugtighedsværdier til heltal.
   int TEMP_INDE_RESULTAT = (Temp_INDE * 10);
   int FUGT_INDE_RESULTAT = (Fugt_INDE * 10);
   int TEMP_UDE_RESULTAT = (Temp_UDE * 10);
   int FUGT_UDE_RESULTAT = (Fugt_UDE * 10);


  //Aflæser Photoresistors (LDR) værdier.
   int LDR_Value = analogRead(lightPin);

  //Hvis RTCen fejler vil den vise "0000:00:00", så den indikerer at rtcen fejler men stadig kører.
  //Hvis RTCen ikke fejler skal hente tiden fra RTC, så man får et rigtigt timestamp.
   String date = rtcFejl ? "0000:00:00": now.timestamp(DateTime::TIMESTAMP_DATE);
   String time = rtcFejl ? "00:00:00" : now.timestamp(DateTime::TIMESTAMP_TIME);

//Tjekker om den aflæste værdi er lig med -999, altså hvis aflæsningen fejler.
//Hvis det er tilfældet at den fejler vil den ikke printe noget.
  if(Temp_INDE == -999 || Fugt_INDE == -999 || Temp_UDE == -999 || Fugt_UDE == -999){

    }
     else{ //elsers skal den køre som normalt.
  //Printer den aflæste data i Serial Monitoren i et txt-format.
   Serial.print(date); Serial.print(","); //Skriver Datoen.
   Serial.print(time); Serial.print(","); //Skriver Tiden.
   Serial.print(TEMP_INDE_RESULTAT); Serial.print("C,"); //Skriver den aflæste Temperatur fra
IndeSensoren, samt viser værdien i grader (C).
```

```
    Serial.print(FUGT_INDE_RESULTAT); Serial.print("%,"); //Skriver den aflæste Fugtighed fra
IndeSensoren, samt viser værdien i Procent (%).
    Serial.print(TEMP_UDE_RESULTAT); Serial.print("C,"); //Skriver den aflæste Temperatur fra
UdeSensoren, samt viser værdien i grader (C).
    Serial.print(FUGT_UDE_RESULTAT) ; Serial.print("%,"); //Skriver den aflæste Fugtighed fra
UdeSensoren, samt viser værdien i procent (%).
    Serial.println(LDR_Value);  //Skriver den aflæste LysIntensitet.

  //Skriver den aflæste data i file "DALO.csv".
    myFile = SD.open("DALO.csv",FILE_WRITE);
    if(myFile){
      myFile.print(date); myFile.print(";");
      myFile.print(time); myFile.print(";");
      myFile.print(TEMP_INDE_RESULTAT); myFile.print("C;");
      myFile.print(FUGT_INDE_RESULTAT); myFile.print("%;");
      myFile.print(TEMP_UDE_RESULTAT); myFile.print("C;");
      myFile.print(FUGT_UDE_RESULTAT); myFile.print("%;");
      myFile.println(LDR_Value);
      myFile.close();
      }
    }
  }
}
```

D. Software Overview

- Biblioteket ”dhtnew.h” bruges til at gøre, så DHT22 sensoren kan afmåle temperatur og fugtighed.

```
#include <dhtnew.h>
```

- Biblioteket ”Wire.h” bruges til at gøre så RTC’en og Arduinoen kan kommunikere ordentlig.

```
#include <Wire.h>
```

- Biblioteket ”RTClib.h” bruges til at hente og danne timestamps.

```
#include <RTClib.h>
```

- Bibliotekerne ”SD.h” og ”SPI.h” bruges for at SD-kortet og Arduinoen kan kommunikere med hinanden.

```
#include <SD.h>
#include <SPI.h>
```

E. Data (.csv) / Excel Table:

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET

09/12/2025	18.16.32	115C	756%	98C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	18.33.40	122C	921%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	18.42.29	119C	891%	106C	1000%	0
09/12/2025	18.52.28	119C	893%	107C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	19.48.46	112C	959%	106C	1000%	0
09/12/2025	20.08.46	110C	819%	100C	1000%	0
09/12/2025	20.18.46	109C	812%	99C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	20.48.46	107C	807%	99C	1000%	0
09/12/2025	20.58.46	107C	806%	99C	1000%	0
09/12/2025	21.08.46	107C	805%	99C	1000%	0
09/12/2025	21.18.46	107C	805%	99C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	21.48.46	107C	805%	99C	1000%	0
09/12/2025	21.58.46	107C	806%	99C	1000%	0
09/12/2025	22.08.46	107C	807%	100C	1000%	0
09/12/2025	22.28.46	107C	807%	100C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	22.58.46	108C	812%	101C	1000%	0

09/12/2025	23.08.46	108C	811%	101C	1000%	0
09/12/2025	23.28.46	108C	811%	102C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	19.38.47	111C	895%	102C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
09/12/2025	23.51.56	112C	870%	104C	1000%	0
10/12/2025	00.21.56	112C	839%	104C	1000%	0
10/12/2025	00.31.56	112C	841%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	00.51.56	113C	842%	106C	1000%	0
10/12/2025	01.01.56	113C	844%	105C	1000%	0
10/12/2025	01.11.56	113C	842%	105C	1000%	0
10/12/2025	01.21.56	113C	843%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	02.01.56	112C	847%	105C	1000%	0
10/12/2025	02.31.56	113C	851%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	02.41.56	113C	852%	106C	1000%	0
10/12/2025	02.51.56	113C	852%	106C	1000%	0
10/12/2025	03.01.56	113C	853%	106C	1000%	0
10/12/2025	03.11.56	113C	854%	106C	1000%	0
10/12/2025	03.31.56	113C	857%	106C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	03.51.56	113C	860%	106C	1000%	0
10/12/2025	04.11.56	114C	867%	106C	1000%	0
10/12/2025	04.31.56	114C	869%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET

10/12/2025	04.51.56	113C	876%	105C	1000%	0
10/12/2025	05.01.56	113C	877%	105C	1000%	0
10/12/2025	05.11.56	113C	881%	104C	1000%	0
10/12/2025	05.21.56	112C	883%	104C	1000%	0
10/12/2025	05.31.56	112C	889%	105C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	06.01.56	112C	899%	104C	1000%	0
10/12/2025	06.11.56	112C	902%	104C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	06.51.56	112C	923%	105C	1000%	0
10/12/2025	07.01.56	112C	929%	105C	1000%	0
10/12/2025	07.31.56	111C	941%	104C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	07.51.56	110C	950%	103C	1000%	0
10/12/2025	08.01.56	110C	953%	103C	1000%	1
10/12/2025	08.11.56	109C	957%	103C	1000%	1
10/12/2025	08.31.56	109C	964%	102C	1000%	38
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	08.51.56	109C	971%	103C	1000%	69
10/12/2025	09.01.56	108C	974%	103C	1000%	78
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	09.41.56	108C	982%	102C	1000%	103
10/12/2025	09.51.56	108C	984%	102C	1000%	146
10/12/2025	10.11.56	108C	987%	103C	1000%	7
10/12/2025	10.31.56	108C	989%	103C	1000%	153
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	10.51.56	108C	991%	102C	1000%	138
10/12/2025	11.01.56	107C	992%	102C	1000%	175

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	11.19.57	112C	969%	107C	1000%	182
10/12/2025	11.29.57	113C	937%	104C	1000%	197
10/12/2025	11.39.57	111C	918%	105C	1000%	225
10/12/2025	11.49.57	110C	903%	105C	1000%	203
10/12/2025	11.59.57	110C	893%	105C	1000%	207
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	12.29.57	110C	874%	104C	1000%	208
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	13.29.57	108C	860%	102C	1000%	172
10/12/2025	13.49.57	107C	858%	101C	1000%	165
10/12/2025	13.59.57	107C	856%	101C	1000%	156
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	14.19.57	106C	858%	101C	1000%	150
10/12/2025	14.29.57	106C	858%	100C	1000%	118
10/12/2025	14.39.57	105C	855%	98C	1000%	124
10/12/2025	14.49.57	105C	854%	98C	1000%	121
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	15.09.57	104C	855%	99C	1000%	81
10/12/2025	15.39.57	103C	854%	96C	1000%	30
10/12/2025	15.49.57	102C	854%	96C	1000%	17
10/12/2025	15.59.57	102C	855%	96C	1000%	6
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	16.29.57	101C	857%	96C	1000%	0
10/12/2025	16.49.57	101C	859%	96C	1000%	0
10/12/2025	16.59.57	101C	860%	96C	1000%	0

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	17.19.57	101C	862%	96C	1000%	0
10/12/2025	17.29.57	101C	863%	97C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	18.19.57	102C	873%	100C	989%	0
10/12/2025	18.49.57	104C	875%	100C	1000%	0
10/12/2025	18.59.57	104C	876%	100C	997%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	19.19.57	104C	877%	101C	991%	0
10/12/2025	19.29.57	105C	878%	101C	988%	0
10/12/2025	19.49.57	104C	877%	100C	997%	0
10/12/2025	19.59.57	104C	880%	100C	998%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	20.19.57	104C	881%	101C	999%	0
10/12/2025	20.29.57	105C	884%	102C	993%	0
10/12/2025	20.39.57	105C	884%	101C	993%	0
10/12/2025	20.49.57	105C	886%	102C	986%	0
10/12/2025	20.59.57	106C	888%	102C	985%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	21.19.57	105C	885%	100C	988%	0
10/12/2025	21.59.57	104C	889%	100C	985%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	22.20.38	106C	968%	103C	988%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	22.40.37	106C	896%	99C	987%	0
10/12/2025	22.50.37	104C	893%	99C	977%	0

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	00.10.37	98C	887%	93C	986%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	00.30.37	97C	885%	91C	993%	0
11/12/2025	00.50.37	97C	886%	92C	984%	0
11/12/2025	01.00.37	97C	886%	93C	979%	0
11/12/2025	01.10.37	97C	886%	93C	976%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	01.50.37	98C	889%	93C	983%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	02.50.37	97C	886%	91C	989%	0
11/12/2025	03.00.37	96C	885%	89C	991%	0
11/12/2025	03.10.37	96C	885%	90C	986%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	03.40.37	94C	886%	91C	978%	0
11/12/2025	03.50.37	94C	886%	90C	979%	0
11/12/2025	04.00.37	95C	887%	91C	971%	0
11/12/2025	04.10.37	95C	885%	89C	974%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	04.30.37	94C	885%	89C	975%	0
11/12/2025	04.40.37	94C	886%	91C	971%	0
11/12/2025	04.50.37	94C	889%	91C	967%	0
11/12/2025	05.00.37	95C	891%	92C	963%	0
11/12/2025	05.10.37	95C	892%	93C	963%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	05.50.37	92C	883%	85C	990%	0
11/12/2025	06.00.37	91C	884%	86C	969%	0
11/12/2025	06.10.37	91C	884%	85C	975%	0

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	06.20.37	91C	885%	88C	961%	0
11/12/2025	06.50.37	92C	888%	86C	981%	0
11/12/2025	07.10.37	90C	883%	84C	998%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	07.30.37	89C	883%	82C	1000%	0
11/12/2025	07.40.37	89C	883%	82C	1000%	0
11/12/2025	07.50.37	88C	884%	83C	1000%	1
11/12/2025	08.00.37	88C	885%	83C	1000%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	08.40.37	91C	893%	88C	998%	34
11/12/2025	08.50.37	92C	894%	88C	998%	41
11/12/2025	09.00.37	92C	894%	89C	997%	41
11/12/2025	09.10.37	92C	896%	90C	992%	54
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	09.40.37	95C	897%	90C	1000%	85
11/12/2025	09.50.37	94C	897%	90C	1000%	116
11/12/2025	10.00.37	95C	898%	91C	1000%	95
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	10.30.37	95C	900%	92C	1000%	143
11/12/2025	10.40.37	95C	901%	92C	1000%	141
11/12/2025	11.00.37	96C	903%	93C	1000%	59
11/12/2025	11.10.37	97C	904%	93C	1000%	155
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	11.20.37	97C	903%	93C	1000%	155
11/12/2025	11.50.37	97C	905%	94C	1000%	113
11/12/2025	12.00.37	97C	905%	93C	1000%	139
11/12/2025	12.10.37	98C	907%	94C	1000%	156

Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
10/12/2025	22.20.38	98C	934%	92C	1000%	93
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	12.57.24	98C	912%	92C	1000%	29
11/12/2025	13.07.24	98C	909%	92C	1000%	33
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	13.47.24	98C	907%	94C	1000%	133
11/12/2025	13.57.24	98C	906%	93C	1000%	89
11/12/2025	14.17.24	98C	907%	93C	997%	68
11/12/2025	14.27.24	98C	907%	93C	999%	56
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	15.07.24	97C	906%	94C	983%	3
11/12/2025	15.17.24	98C	906%	94C	985%	20
11/12/2025	15.27.24	98C	906%	94C	982%	1
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	15.47.24	97C	905%	93C	985%	1
11/12/2025	15.57.24	97C	905%	93C	983%	0
11/12/2025	16.17.24	97C	904%	92C	990%	0
11/12/2025	16.27.24	96C	905%	91C	989%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	16.47.24	95C	905%	91C	988%	0
11/12/2025	16.57.24	95C	905%	91C	980%	0
11/12/2025	17.27.24	96C	907%	92C	973%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	17.47.24	96C	907%	93C	963%	0
11/12/2025	18.07.24	96C	907%	93C	949%	0

11/12/2025	18.27.24	97C	906%	93C	948%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	18.47.24	97C	906%	92C	953%	0
11/12/2025	18.57.24	96C	906%	92C	962%	0
11/12/2025	19.17.24	96C	906%	91C	968%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	19.37.24	95C	905%	90C	974%	0
11/12/2025	19.47.24	95C	906%	91C	974%	0
11/12/2025	19.57.24	95C	906%	91C	969%	0
11/12/2025	20.07.24	95C	905%	91C	959%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	20.27.39	98C	928%	97C	934%	0
11/12/2025	20.47.39	96C	908%	92C	946%	0
11/12/2025	21.07.39	95C	909%	92C	952%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	21.37.39	95C	906%	89C	963%	0
11/12/2025	21.57.39	94C	906%	89C	968%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	22.27.39	94C	906%	89C	966%	0
11/12/2025	22.37.39	94C	905%	89C	961%	0
11/12/2025	22.47.39	94C	905%	89C	961%	0
11/12/2025	22.57.39	93C	904%	88C	968%	0
11/12/2025	23.07.39	93C	902%	87C	975%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
11/12/2025	23.27.39	92C	904%	88C	977%	0
11/12/2025	23.37.39	92C	904%	88C	978%	0

11/12/2025	23.57.39	92C	904%	88C	982%	0
12/12/2025	00.07.39	92C	903%	87C	982%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	00.27.39	92C	903%	87C	984%	0
12/12/2025	00.37.39	92C	904%	88C	980%	0
12/12/2025	00.47.39	92C	904%	88C	980%	0
12/12/2025	00.57.39	93C	905%	89C	976%	0
12/12/2025	01.07.39	93C	906%	89C	976%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	01.17.39	93C	906%	89C	976%	0
12/12/2025	01.27.39	93C	906%	88C	980%	0
12/12/2025	01.37.39	93C	905%	89C	981%	0
12/12/2025	01.47.39	93C	905%	88C	982%	0
12/12/2025	01.57.39	93C	906%	89C	981%	0
12/12/2025	02.07.39	93C	906%	89C	978%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	02.27.39	93C	907%	90C	970%	0
12/12/2025	02.47.39	94C	908%	91C	959%	0
12/12/2025	03.07.39	94C	908%	91C	957%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	03.27.39	95C	908%	91C	955%	0
12/12/2025	03.37.39	95C	908%	91C	954%	0
12/12/2025	03.47.39	95C	908%	91C	955%	0
12/12/2025	04.07.39	95C	908%	91C	957%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	04.27.39	94C	907%	90C	959%	0
12/12/2025	04.47.39	94C	907%	90C	964%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET

12/12/2025	05.37.39	93C	907%	88C	982%	0
12/12/2025	05.47.39	93C	907%	87C	982%	0
12/12/2025	05.57.39	93C	906%	87C	981%	0
12/12/2025	06.07.39	92C	907%	87C	983%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	06.27.39	92C	907%	87C	979%	0
12/12/2025	06.37.39	92C	907%	87C	977%	0
12/12/2025	06.47.39	92C	907%	87C	978%	0
12/12/2025	06.57.39	91C	906%	86C	977%	0
12/12/2025	07.07.39	91C	906%	86C	973%	0
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET
12/12/2025	07.27.39	90C	905%	85C	959%	0
12/12/2025	07.37.39	90C	905%	86C	957%	0
12/12/2025	07.47.39	90C	905%	86C	941%	0
12/12/2025	08.07.39	89C	906%	85C	951%	7
Dato	Tid	Temp_INDE	Fugt_INDE	Temp_UDE	Fugt_UDE	Lys_INTENSITET